

AAE 590 LGM HW4

Yahor Lechanka

April 2026

1 The Banana Distribution

The std for the velocity noise was changed to 0.1.

1.1 Simulate the unicycle.

See Appendix for the code.

1.2 Plot the distribution.

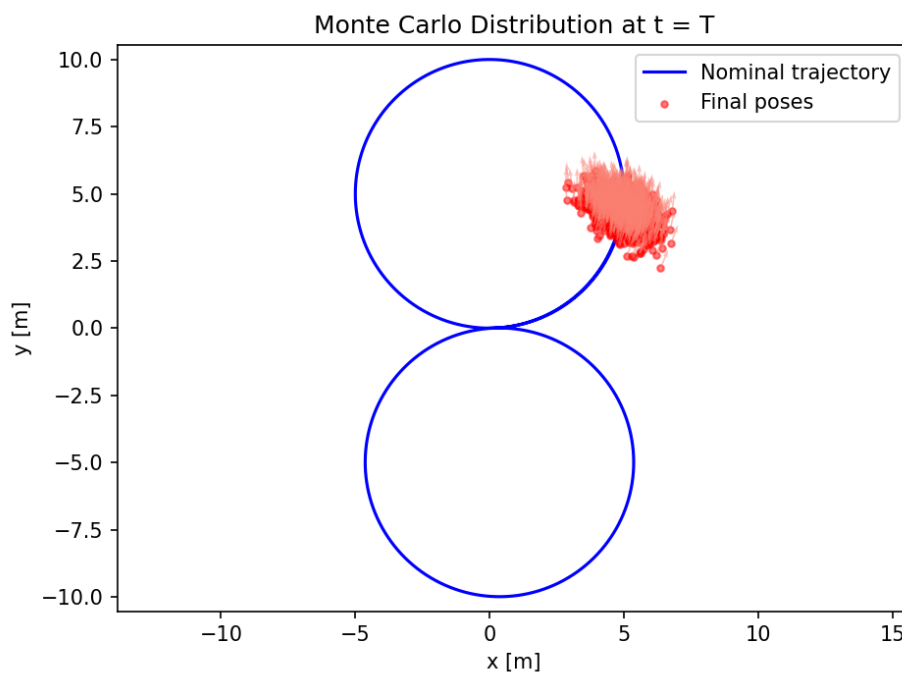


Figure 1: Nominal trajectory and Monte Carlo distribution.

1.3 Fit a Gaussian.

$$\bar{p} = \begin{bmatrix} 4.9350 \\ 4.2802 \end{bmatrix} \text{ m}$$
$$S = \begin{bmatrix} 2.1308 & 0.0429 \\ 0.0429 & 2.1889 \end{bmatrix} \text{ m}^2$$

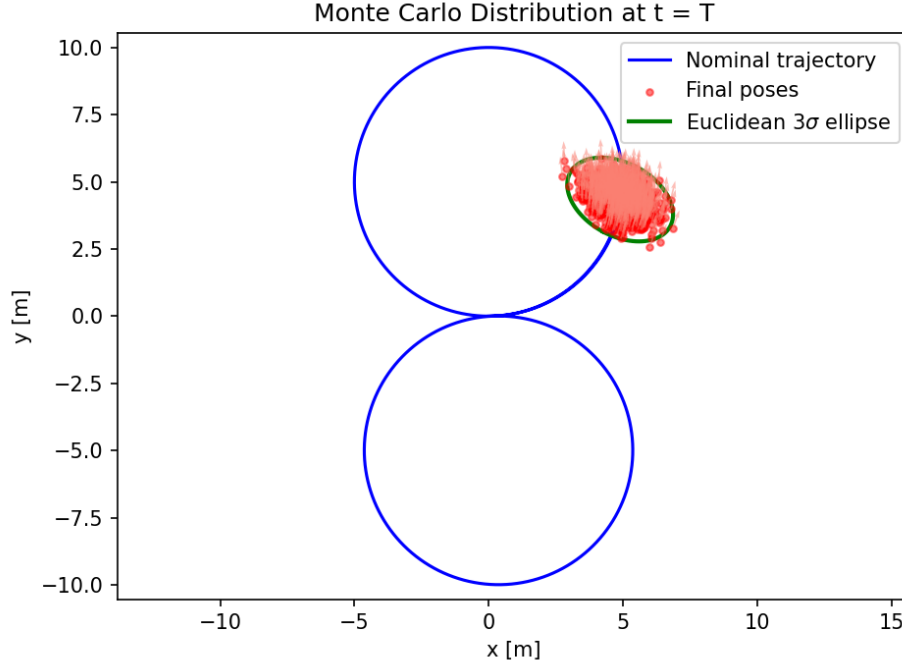


Figure 2: Gaussian Fit

The Gaussian fits the shape of the point cloud.

1.4 Characterize the distribution.

The overall shape looks like an ellipse with maybe slightly deformation. This is because we only have noise in the vehicle forward direction and combining it with the movement along the arc, where we have angular velocity and its uncertainty, it accumulates the error that produces a banana shape. The Gaussian ellipse cannot properly approximate it because it assumes symmetric distribution around mean which is not true in our case.

1.5 Intermediate snapshots.

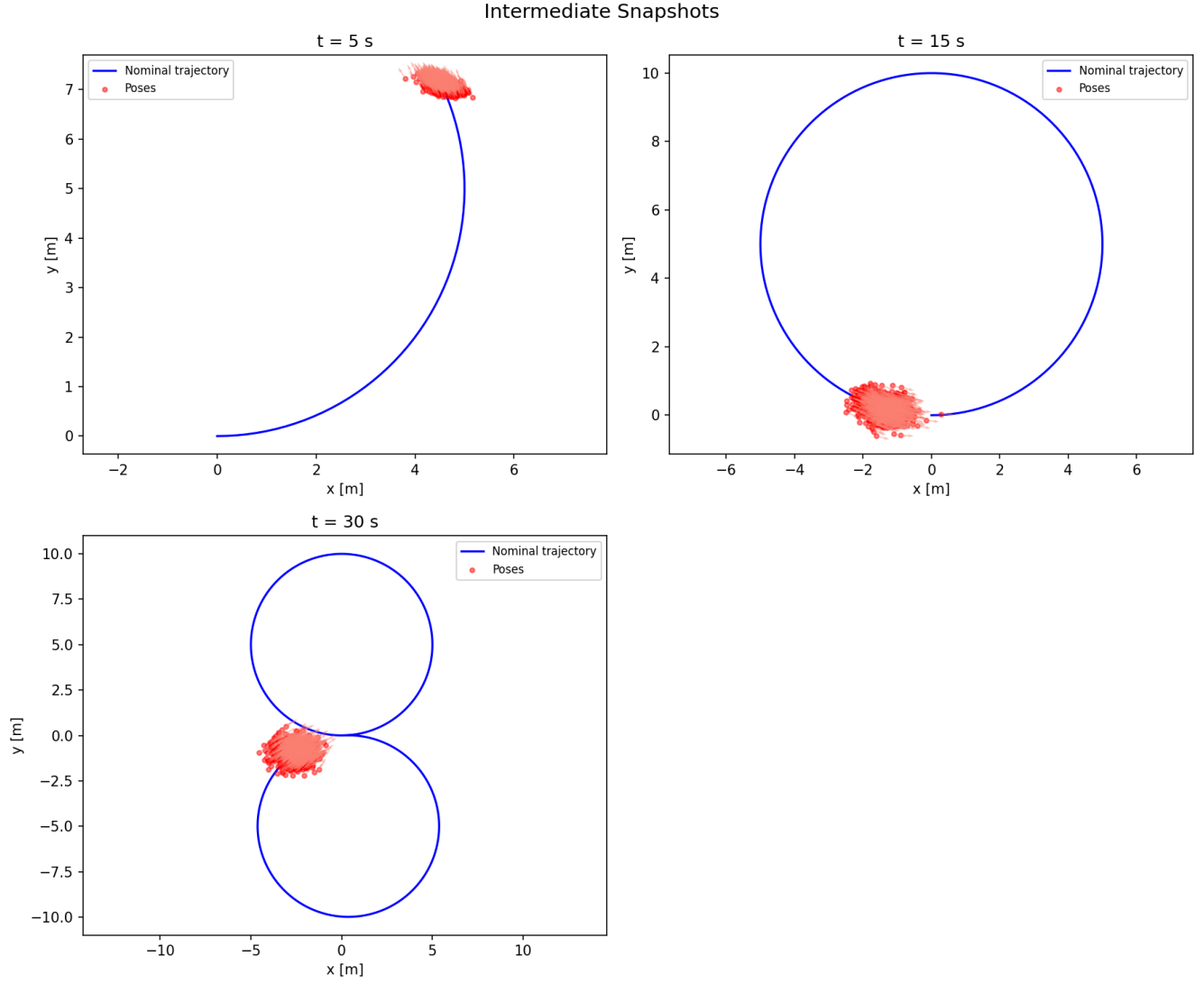


Figure 3: Snapshots

At $t = 5$ s we can see an oval shape and at other snapshots the distributions gets more dispersed.

1.6 Lie algebra Gaussian fit.

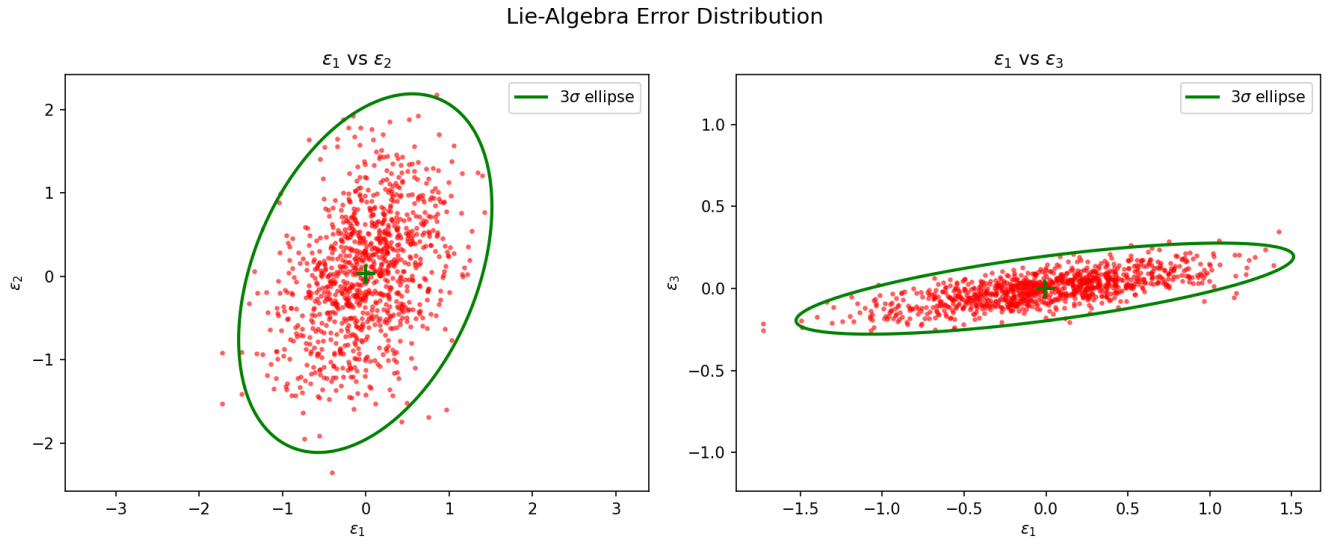


Figure 4: Gaussian Distribution In Lie algebra

$$\bar{\epsilon} = \begin{bmatrix} -0.0338 \\ -0.0459 \\ -0.00085 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 2.0661 & -0.0299 & 0.0314 \\ -0.0299 & 2.3565 & 0.0427 \\ 0.0314 & 0.0427 & 0.00806 \end{bmatrix}$$

The clouds in both projections are approximately ellipsoidal.

1.7 Exponentiate the ellipsoid boundary.

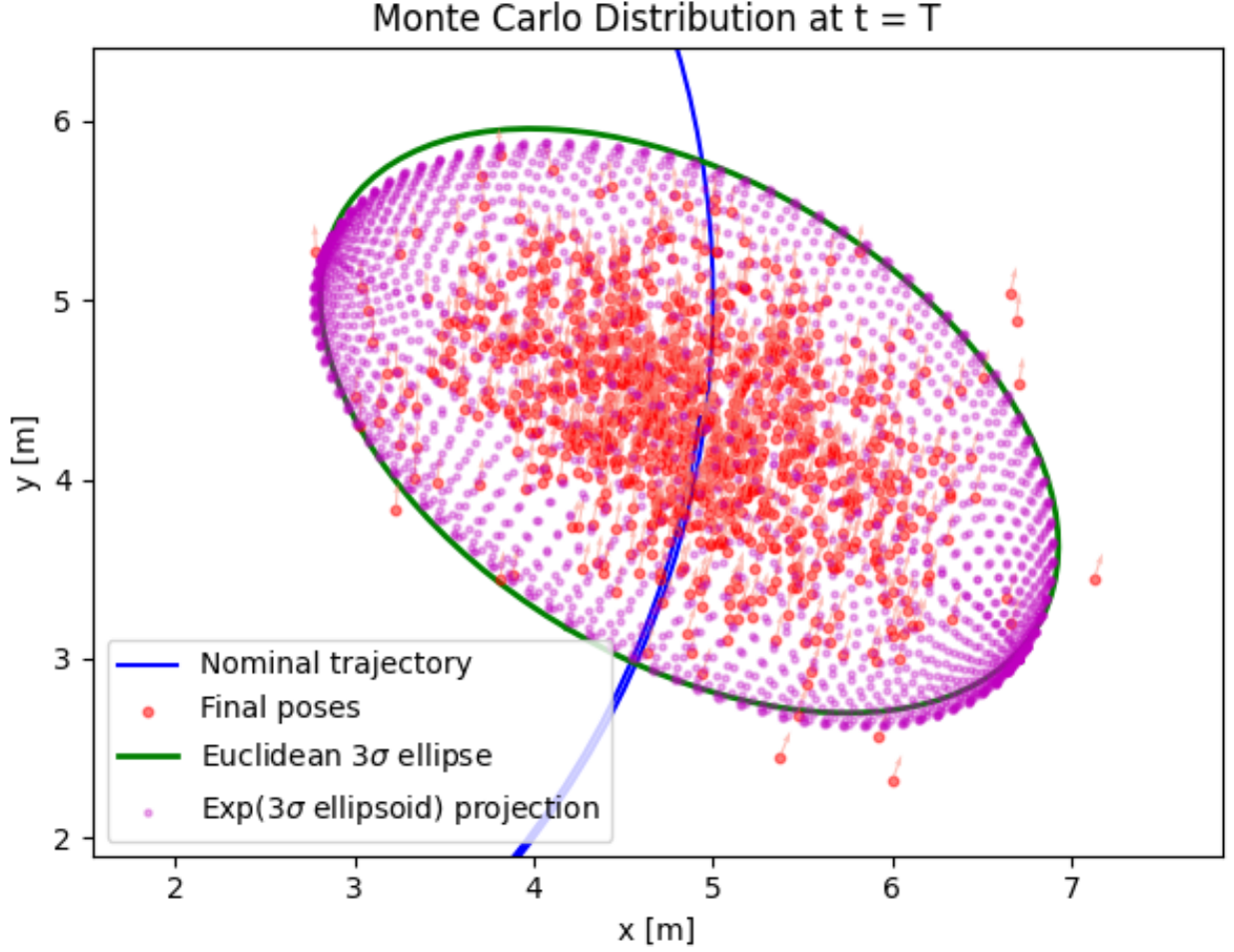


Figure 5: Lie algebra ellipsoid projection

We can see some deviation from an ideal ellipsoid which resembles the banana shape. A straight ellipsoid in the Lie algebra maps to a curved shape because the exponential map is non linear because the heading component couples into translational part.

2 World-Frame EKF

2.1 Prediction step.

$$F_k = \begin{bmatrix} 1 & 0 & -\Delta t v_{\text{nom}} \sin(\theta) \\ 0 & 1 & \Delta t v_{\text{nom}} \cos(\theta) \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} Q_k^{(\text{world})} &= \mathbb{E}[(L_k \mathbf{w})(L_k \mathbf{w})^\top] \\ &= \mathbb{E}[L_k \mathbf{w} \mathbf{w}^\top L_k^\top] \\ &= L_k \mathbb{E}[\mathbf{w} \mathbf{w}^\top] L_k^\top \\ &= L_k Q_{\text{body}} L_k^\top. \end{aligned}$$

where $L_k = \Delta t \begin{bmatrix} \cos \hat{\theta}_k & -\sin \hat{\theta}_k & 0 \\ \sin \hat{\theta}_k & \cos \hat{\theta}_k & 0 \\ 0 & 0 & 1 \end{bmatrix}$

But because we have to lateral slip, we have no noise there and the second column is zero.

$$L_k = \Delta t \begin{bmatrix} \cos \hat{\theta}_k & 0 & 0 \\ \sin \hat{\theta}_k & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The covariance rotates because the noise is attached to the body frame, but our filter works in the world frame, so we have to rotate the body-frame covariance into world coordinates. Since this rotation depends on the current heading $\hat{\theta}_k$, which changes every timestep, the world-frame covariance reorients accordingly.

2.2 Measurement update.

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

2.3 Implementation

See Appendix for the code.

2.4 Animation.

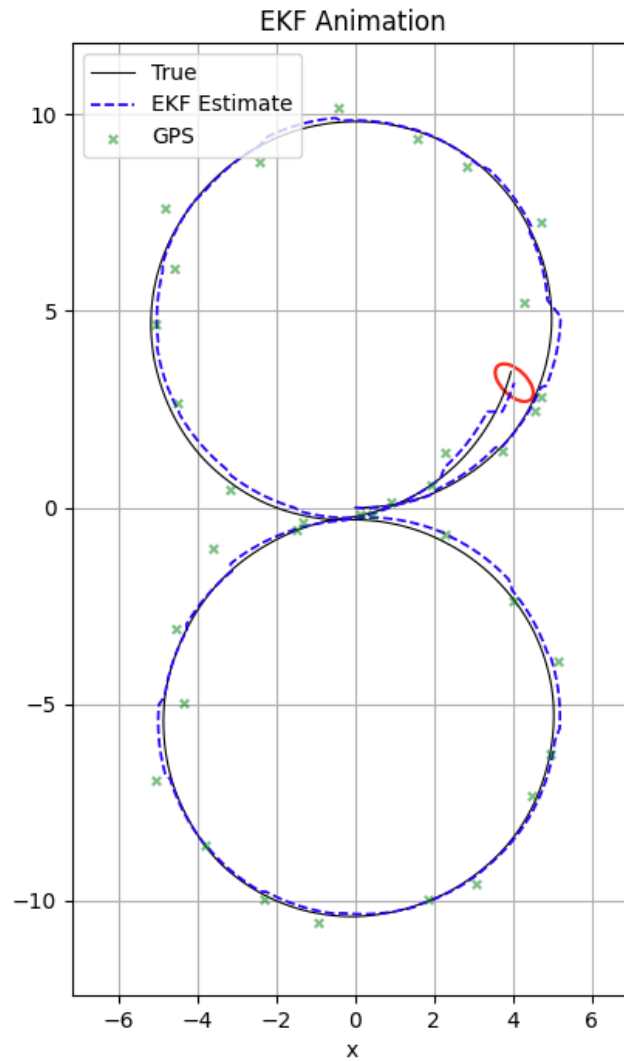


Figure 6: Unicycle with EKF

The long axis of the covariance ellipse is never aligned with the trajectory of the unicycle. It is close to be perpendicular to it. However if we increase the std of velocity to 1 m/s, the long axis is almost always aligned with the trajectory.

2.5 Monte Carlo evaluation.

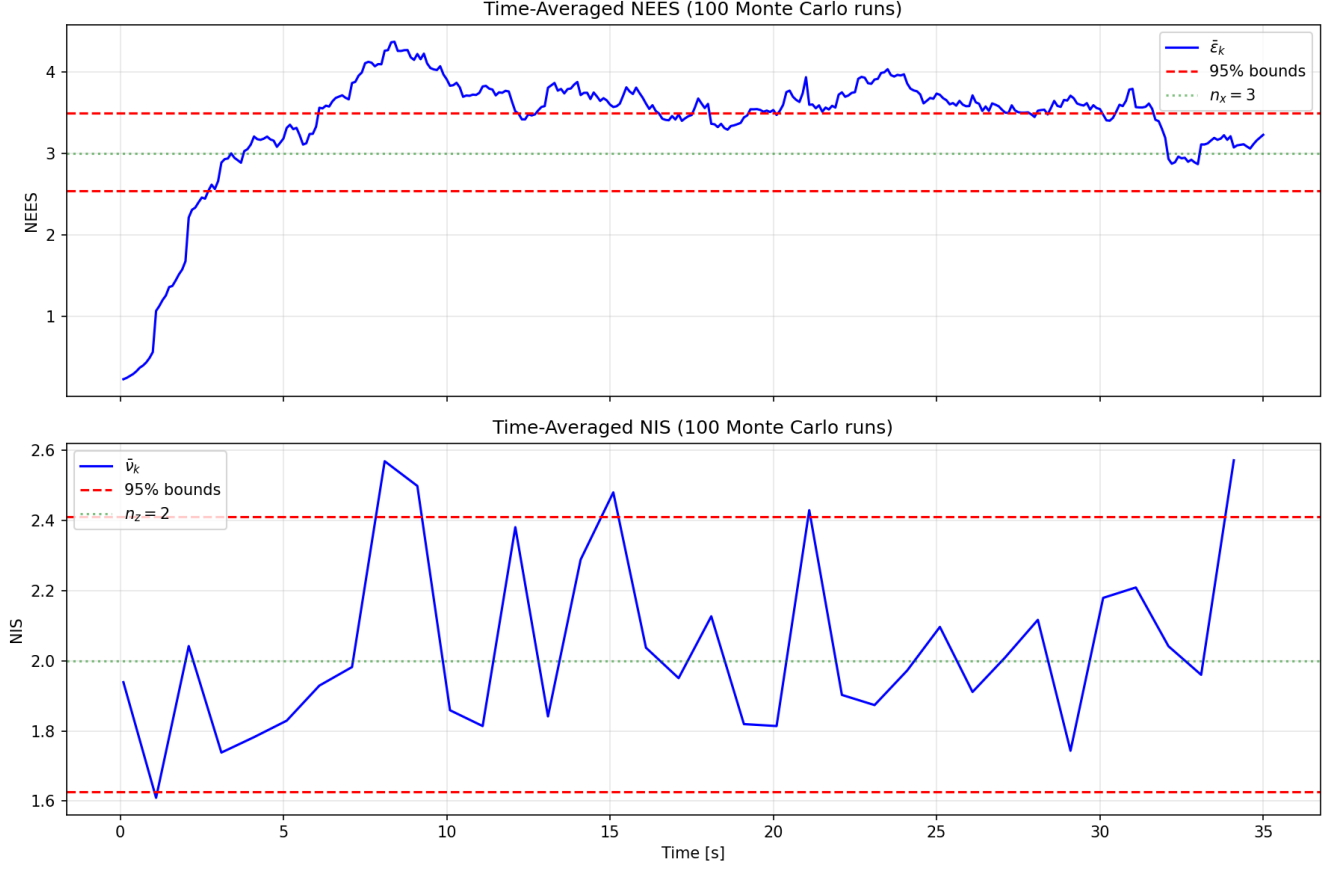


Figure 7: Time averages NEES and NIS

The NEES is consistently above 95% line which suggests that our filter is overconfident. It also has low values in the first 3 seconds, which indicates that our filter is under-confident with the initial value of P .

The NIS stays stable throughout the run and averages around 2, which is the measurement dimension. The advantage of this metric is that we can calculate it without the true state, which means we can monitor the filter's performance onboard the controller.

3 Error-State EKF in the Body Frame

3.1 Error dynamics.

1. Position differences (world frame)

True and reference dynamics for x :

$$\begin{aligned} x_{k+1} &= x_k + (v_{\text{nom}} + w_v) \cos \theta_k \Delta t, \\ \hat{x}_{k+1} &= \hat{x}_k + v_{\text{nom}} \cos \hat{\theta}_k \Delta t. \end{aligned}$$

Subtracting,

$$x_{k+1} - \hat{x}_{k+1} = (x_k - \hat{x}_k) + v_{\text{nom}} \Delta t (\cos \theta_k - \cos \hat{\theta}_k) + w_v \Delta t \cos \theta_k.$$

Expanding $\cos \theta_k = \cos(\hat{\theta}_k + \delta\theta)$ to first order:

$$\cos \theta_k \approx \cos \hat{\theta}_k \underbrace{\cos \delta\theta}_{\approx 1} - \sin \hat{\theta}_k \underbrace{\sin \delta\theta}_{\approx \delta\theta} = \cos \hat{\theta}_k - \delta\theta \sin \hat{\theta}_k,$$

so $\cos \theta_k - \cos \hat{\theta}_k \approx -\delta\theta \sin \hat{\theta}_k$, giving

$$x_{k+1} - \hat{x}_{k+1} \approx (x_k - \hat{x}_k) - v_{\text{nom}} \Delta t \delta\theta \sin \hat{\theta}_k + w_v \Delta t \cos \hat{\theta}_k.$$

Analogously for y :

$$\begin{aligned} y_{k+1} &= y_k + (v_{\text{nom}} + w_v) \sin \theta_k \Delta t, \\ \hat{y}_{k+1} &= \hat{y}_k + v_{\text{nom}} \sin \hat{\theta}_k \Delta t, \\ \sin \theta_k &\approx \sin \hat{\theta}_k + \delta\theta \cos \hat{\theta}_k, \\ y_{k+1} - \hat{y}_{k+1} &\approx (y_k - \hat{y}_k) + v_{\text{nom}} \Delta t \delta\theta \cos \hat{\theta}_k + w_v \Delta t \sin \hat{\theta}_k. \end{aligned}$$

2. Expand trig at $\hat{\theta}_{k+1}$

Since $\hat{\theta}_{k+1} = \hat{\theta}_k + \omega_{\text{ref}} \Delta t$,

$$\begin{aligned} \cos \hat{\theta}_{k+1} &\approx \cos \hat{\theta}_k - \sin \hat{\theta}_k \omega_{\text{ref}} \Delta t, \\ \sin \hat{\theta}_{k+1} &\approx \sin \hat{\theta}_k + \cos \hat{\theta}_k \omega_{\text{ref}} \Delta t. \end{aligned}$$

3. Body-frame error δx_b^{k+1}

Let $C = \cos \hat{\theta}_k$, $S = \sin \hat{\theta}_k$ for brevity. Then

$$\delta x_b^{k+1} = \cos \hat{\theta}_{k+1} (x_{k+1} - \hat{x}_{k+1}) + \sin \hat{\theta}_{k+1} (y_{k+1} - \hat{y}_{k+1}).$$

Substituting everything and labeling terms:

$$\begin{aligned} \delta x_b^{k+1} &= (C - S \omega_{\text{ref}} \Delta t) \left[\underbrace{(x_k - \hat{x}_k)}_{(1)} - \underbrace{v_{\text{nom}} \Delta t \delta\theta S}_{(2)} + \underbrace{w_v \Delta t C}_{(3)} \right] \\ &\quad + (S + C \omega_{\text{ref}} \Delta t) \left[\underbrace{(y_k - \hat{y}_k)}_{(4)} + \underbrace{v_{\text{nom}} \Delta t \delta\theta C}_{(5)} + \underbrace{w_v \Delta t S}_{(6)} \right]. \end{aligned}$$

Collect terms, dropping any product of two small quantities (two of $\delta\theta, \delta x_b, \delta y_b, w_v, w_\omega, \Delta t$ beyond first order):

$$\begin{aligned} C \cdot (1) + S \cdot (4) &= C(x_k - \hat{x}_k) + S(y_k - \hat{y}_k) = \delta x_b^k, \\ -S \omega_{\text{ref}} \Delta t \cdot (1) + C \omega_{\text{ref}} \Delta t \cdot (4) &= \omega_{\text{ref}} \Delta t [-S(x_k - \hat{x}_k) + C(y_k - \hat{y}_k)] = \omega_{\text{ref}} \Delta t \delta y_b^k, \\ C \cdot (2) + S \cdot (5) &= -v_{\text{nom}} \Delta t \delta\theta CS + v_{\text{nom}} \Delta t \delta\theta SC = 0, \\ C \cdot (3) + S \cdot (6) &= w_v \Delta t C^2 + w_v \Delta t S^2 = w_v \Delta t, \end{aligned}$$

with all remaining cross-terms $\mathcal{O}(\Delta t^2)$ or products of two small quantities. Therefore:

$$\boxed{\delta x_b^{k+1} \approx \delta x_b^k + \omega_{\text{ref}} \Delta t \delta y_b^k + w_v \Delta t.}$$

4. Body-frame error δy_b^{k+1}

$$\delta y_b^{k+1} = -\sin \hat{\theta}_{k+1} (x_{k+1} - \hat{x}_{k+1}) + \cos \hat{\theta}_{k+1} (y_{k+1} - \hat{y}_{k+1}).$$

Using the same expansion,

$$\begin{aligned} \delta y_b^{k+1} &= -(S + C \omega_{\text{ref}} \Delta t) \left[(x_k - \hat{x}_k) - v_{\text{nom}} \Delta t \delta\theta S + w_v \Delta t C \right] \\ &\quad + (C - S \omega_{\text{ref}} \Delta t) \left[(y_k - \hat{y}_k) + v_{\text{nom}} \Delta t \delta\theta C + w_v \Delta t S \right]. \end{aligned}$$

Collecting:

$$\begin{aligned}
-S(x_k - \hat{x}_k) + C(y_k - \hat{y}_k) &= \delta y_b^k, \\
-C\omega_{\text{ref}}\Delta t(x_k - \hat{x}_k) - S\omega_{\text{ref}}\Delta t(y_k - \hat{y}_k) &= -\omega_{\text{ref}}\Delta t \delta x_b^k, \\
v_{\text{nom}}\Delta t \delta\theta(S^2 + C^2) &= v_{\text{nom}}\Delta t \delta\theta, \\
w_v\Delta t(-SC + CS) &= 0.
\end{aligned}$$

Therefore:

$$\delta y_b^{k+1} \approx -\omega_{\text{ref}}\Delta t \delta x_b^k + \delta y_b^k + v_{\text{nom}}\Delta t \delta\theta.$$

5. Heading error

$$\theta_{k+1} - \hat{\theta}_{k+1} = \delta\theta + w_\omega\Delta t \implies \delta\theta^{k+1} = \delta\theta + w_\omega\Delta t.$$

6. Assembled linearization

$$\varepsilon_{k+1} \approx A_k \varepsilon_k + G n_k,$$

with

$$A_k = \begin{bmatrix} 1 & \omega_{\text{ref}}\Delta t & 0 \\ -\omega_{\text{ref}}\Delta t & 1 & v_{\text{nom}}\Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad G = \begin{bmatrix} \Delta t & 0 & 0 \\ 0 & \Delta t & 0 \\ 0 & 0 & \Delta t \end{bmatrix} = \Delta t I_3, \quad n_k = \begin{bmatrix} w_v \\ 0 \\ w_\omega \end{bmatrix}.$$

A_k depends only on the commanded inputs $(v_{\text{nom}}, \omega_{\text{ref}})$ and not on the reference pose $(\hat{x}_k, \hat{y}_k, \hat{\theta}_k)$. Since we don't have to rotate out covariance matrix anymore, the covariance prediction is then

$$P_{k+1|k} = A_k P_{k|k} A_k^\top + G Q_{\text{body}} G^\top.$$

3.2 Observation: process noise is natural.

In error-state Kalman Filter the state (which is error) is defined in body frame. The covariance matrix represents noise for the body frame vectors, so no rotation is required since it is already in the body frame.

3.3 Measurement update.

$$\begin{aligned}
z_k &= [x_k^w, y_k^w]^T + v_k \\
[x_k^w, y_k^w]^T &= [\hat{x}_k^w, \hat{y}_k^w]^T + [\delta x_k^w, \delta y_k^w]^T \\
[\delta x_k^w, \delta y_k^w]^T &= \hat{R}_k [\delta x_k^b, \delta y_k^b]^T \\
[x_k^w, y_k^w]^T &= [\hat{x}_k^w, \hat{y}_k^w]^T + \hat{R}_k [\delta x_k^b, \delta y_k^b]^T \\
z_k &\approx \hat{p}_k + \hat{R}_k [\delta x_k^b, \delta y_k^b]^T + v_k
\end{aligned}$$

$$\text{So, } H_k = [\hat{R}_k \quad \mathbf{0}_2]$$

3.4 Implementation.

See Appendix for the code.

3.5 Animation and comparison.

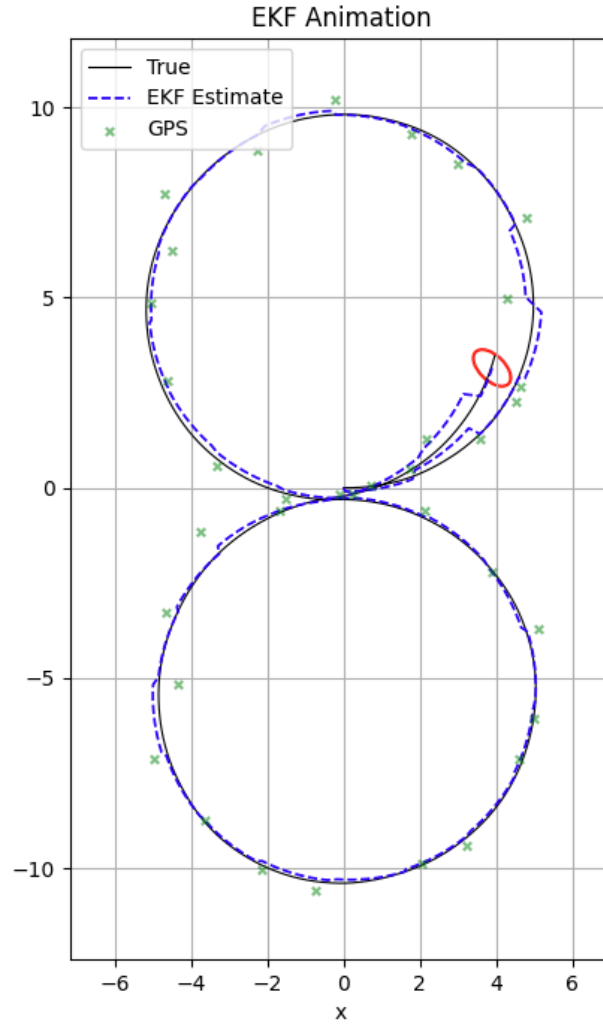


Figure 8: Unicycle with ES-EKF

We can see similar behavior as for the world frame EKF. For small std v , the long axis of the ellipse is never aligned with the trajectory, but for large std v the long axis is aligned with the trajectory. This might be a better match for banana distribution because the error is defined in body frame.

3.6 NEES and NIS comparison.

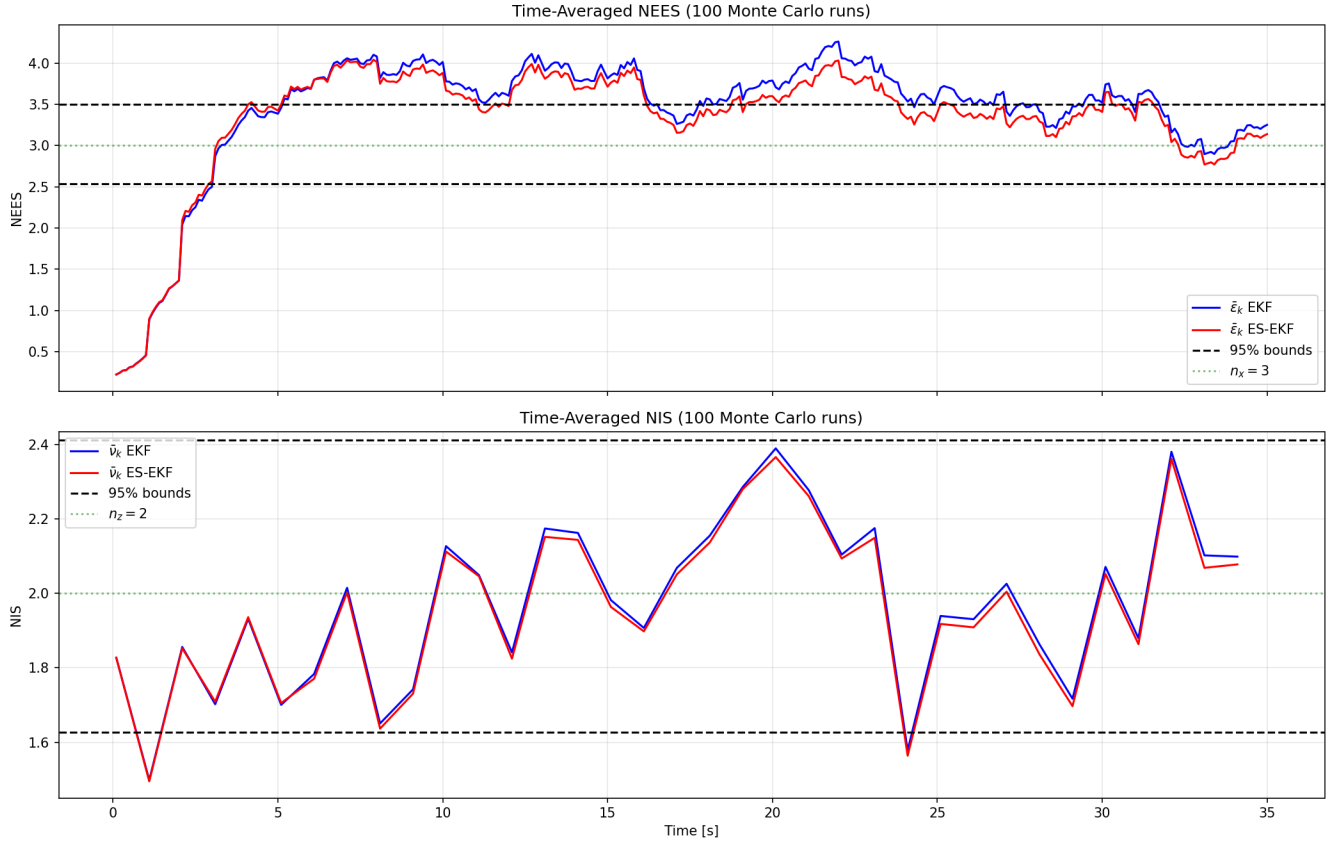


Figure 9: EKF vs ES-EKF

For NEES, the ES-EKF stays closer to the state dimension of 3. For NIS, ES-EKF is slightly better, but both filters stay around 2.

4 Left-Invariant IEKF

4.1 Left-invariant error dynamics via Lie algebra.

$$\begin{aligned}
\eta_k &\approx \exp(\varepsilon_k^\wedge), \quad \varepsilon_k = \log(\eta_k)^\vee \\
\hat{X}_{k+1} &= \hat{X}_k \exp(\hat{\xi}^\wedge \Delta t) \quad \hat{X}_{k+1}^{-1} = \exp(\hat{\xi}^\wedge \Delta t)^{-1} \hat{X}_k^{-1} \quad X_{k+1} = X_k \exp((\hat{\xi} + n_k)^\wedge \Delta t) \\
\varepsilon_{k+1} &= \\
&= \log(\eta_{k+1}) \\
&= \log(\hat{X}_{k+1}^{-1} X_{k+1}) \\
&= \log\left(\exp(\hat{\xi}^\wedge \Delta t)^{-1} \hat{X}_k^{-1} X_k \exp((\hat{\xi} + n_k)^\wedge \Delta t)\right) \\
&= \log\left(\exp(-\hat{\xi}^\wedge \Delta t) \hat{X}_k^{-1} X_k \exp(\hat{\xi}^\wedge \Delta t) \exp(n_k^\wedge \Delta t)\right) \\
&= \log\left(\exp(-\hat{\xi}^\wedge \Delta t) \exp(\varepsilon_k^\wedge) \exp(\hat{\xi}^\wedge \Delta t)\right) + \log(\exp(n_k^\wedge \Delta t)) \\
&= \log\left(\exp\left(\underbrace{\text{Ad}_{\exp(-\hat{\xi}^\wedge \Delta t)} \varepsilon_k^\wedge}_{\text{Ad}_{\exp(-\hat{\xi}^\wedge \Delta t)}}\right) + n_k \Delta t\right) \\
&= \text{Ad}_{\exp(-\hat{\xi}^\wedge \Delta t)} \varepsilon_k + n_k \Delta t \\
&= A_k \varepsilon_k + n_k \Delta t
\end{aligned}$$

$$A_k = \text{Ad}_{\exp(-\hat{\xi}^\wedge \Delta t)}.$$

$$A_c = \left. \frac{d}{d\Delta t} \text{Ad}_{\exp(-\hat{\xi}^\wedge \Delta t)} \right|_{\Delta t=0}$$

$$\text{Ad}_{\exp(-\hat{\xi}^\wedge \Delta t)}(\varepsilon^\wedge) = \exp(-\hat{\xi}^\wedge \Delta t) \varepsilon^\wedge \exp(\hat{\xi}^\wedge \Delta t)$$

Differentiating in Δt by the product rule:

$$\frac{d}{d\Delta t} \left[\exp(-\hat{\xi}^\wedge \Delta t) \varepsilon^\wedge \exp(\hat{\xi}^\wedge \Delta t) \right] = -\hat{\xi}^\wedge \exp(-\hat{\xi}^\wedge \Delta t) \varepsilon^\wedge \exp(\hat{\xi}^\wedge \Delta t) + \exp(-\hat{\xi}^\wedge \Delta t) \varepsilon^\wedge \hat{\xi}^\wedge \exp(\hat{\xi}^\wedge \Delta t).$$

Evaluating at $\Delta t = 0$ (so $\exp(0) = I$)

$$\left. \frac{d}{d\Delta t} \text{Ad}_{\exp(-\hat{\xi}^\wedge \Delta t)}(\varepsilon^\wedge) \right|_{\Delta t=0} = -\hat{\xi}^\wedge \varepsilon^\wedge + \varepsilon^\wedge \hat{\xi}^\wedge = -[\hat{\xi}^\wedge, \varepsilon^\wedge] = -\text{ad}_{\hat{\xi}}(\varepsilon^\wedge)$$

Hence $A_c = -\text{ad}_{\hat{\xi}}$

Computing $\text{ad}_{\hat{\xi}}$ by the commutator formula.

With $\hat{\xi} = [\hat{v}, 0, \hat{\omega}]^\top$ and $\varepsilon = [\delta x_b, \delta y_b, \delta \theta]^\top$, the hat maps are

$$\begin{aligned}
\hat{\xi}^\wedge &= \begin{bmatrix} 0 & -\hat{\omega} & \hat{v} \\ \hat{\omega} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \varepsilon^\wedge = \begin{bmatrix} 0 & -\delta \theta & \delta x_b \\ \delta \theta & 0 & \delta y_b \\ 0 & 0 & 0 \end{bmatrix}. \\
\hat{\xi}^\wedge \varepsilon^\wedge &= \begin{bmatrix} -\hat{\omega} \delta \theta & 0 & -\hat{\omega} \delta y_b \\ 0 & -\hat{\omega} \delta \theta & \hat{\omega} \delta x_b \\ 0 & 0 & 0 \end{bmatrix}, \quad \varepsilon^\wedge \hat{\xi}^\wedge = \begin{bmatrix} -\hat{\omega} \delta \theta & 0 & 0 \\ 0 & -\hat{\omega} \delta \theta & \delta \theta \hat{v} \\ 0 & 0 & 0 \end{bmatrix}. \\
[\hat{\xi}^\wedge, \varepsilon^\wedge] &= \begin{bmatrix} 0 & 0 & -\hat{\omega} \delta y_b \\ 0 & 0 & \hat{\omega} \delta x_b - \hat{v} \delta \theta \\ 0 & 0 & 0 \end{bmatrix},
\end{aligned}$$

$$\text{ad}_{\hat{\xi}} \varepsilon = \begin{bmatrix} -\hat{\omega} \delta y_b \\ \hat{\omega} \delta x_b - \hat{v} \delta \theta \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & -\hat{\omega} & 0 \\ \hat{\omega} & 0 & -\hat{v} \\ 0 & 0 & 0 \end{bmatrix} \varepsilon.$$

Therefore

$$A_c = -\text{ad}_{\hat{\xi}} = \begin{bmatrix} 0 & \hat{\omega} & 0 \\ -\hat{\omega} & 0 & \hat{v} \\ 0 & 0 & 0 \end{bmatrix}.$$

As we can see, both A_k and A_c depend only on the error and not the state.

$$P_{k+1|k} = A_k P_{k|k} A_k^\top + G Q_{\text{body}} G^\top.$$

4.2 Measurement Jacobian.

The measurement function h extracts the translation part of $X \in SE(2)$. Write $\hat{X}_k$ and the perturbation in block form:

$$\hat{X}_k = \begin{bmatrix} \hat{R}_k & \hat{p}_k \\ 0 & 1 \end{bmatrix}, \quad \text{Exp}(\varepsilon^\wedge) = \begin{bmatrix} R(\delta\theta) & V(\delta\theta) \begin{bmatrix} \delta x_b \\ \delta y_b \end{bmatrix} \\ 0 & 1 \end{bmatrix},$$

where $V(\delta\theta)$ is the $SE(2)$ left Jacobian. To first order in ε ,

$$R(\delta\theta) \approx I + \delta\theta J, \quad V(\delta\theta) \approx I, \quad J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix},$$

so

$$\text{Exp}(\varepsilon^\wedge) \approx \begin{bmatrix} I + \delta\theta J & \begin{bmatrix} \delta x_b \\ \delta y_b \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

Multiplying,

$$\hat{X}_k \text{Exp}(\varepsilon^\wedge) = \begin{bmatrix} \hat{R}_k(I + \delta\theta J) & \hat{p}_k + \hat{R}_k \begin{bmatrix} \delta x_b \\ \delta y_b \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

The measurement h picks off the translation block:

$$h(\hat{X}_k \text{Exp}(\varepsilon^\wedge)) = \hat{p}_k + \hat{R}_k \begin{bmatrix} \delta x_b \\ \delta y_b \end{bmatrix}$$

$$z_k = \hat{p}_k + \hat{R}_k \begin{bmatrix} \delta x_b \\ \delta y_b \end{bmatrix} + v_k$$

So

$$H_k = [\hat{R}_k \quad \mathbf{0}_2] \in \mathbb{R}^{2 \times 3}.$$

4.3 Implementation.

See Appendix for the code.

4.4 Full comparison.

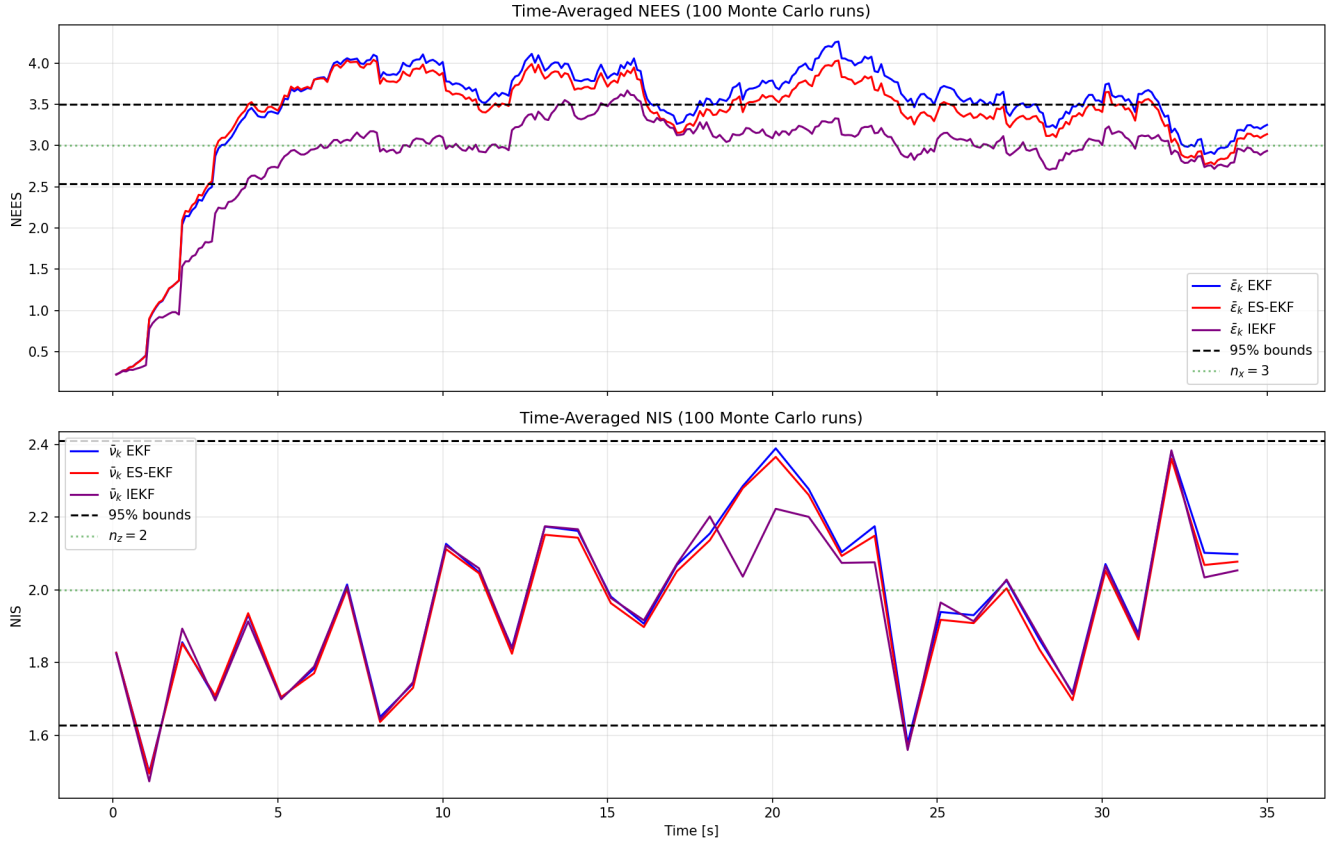


Figure 10: Filter comparison

Filter	Time-averaged NEES	Time-averaged NIS
World-frame EKF	3.4338	1.9709
Error-state EKF	3.3302	1.9567
Left-invariant IEKF	2.8742	1.9509

Table 1: Filter consistency comparison over 100 Monte Carlo runs.

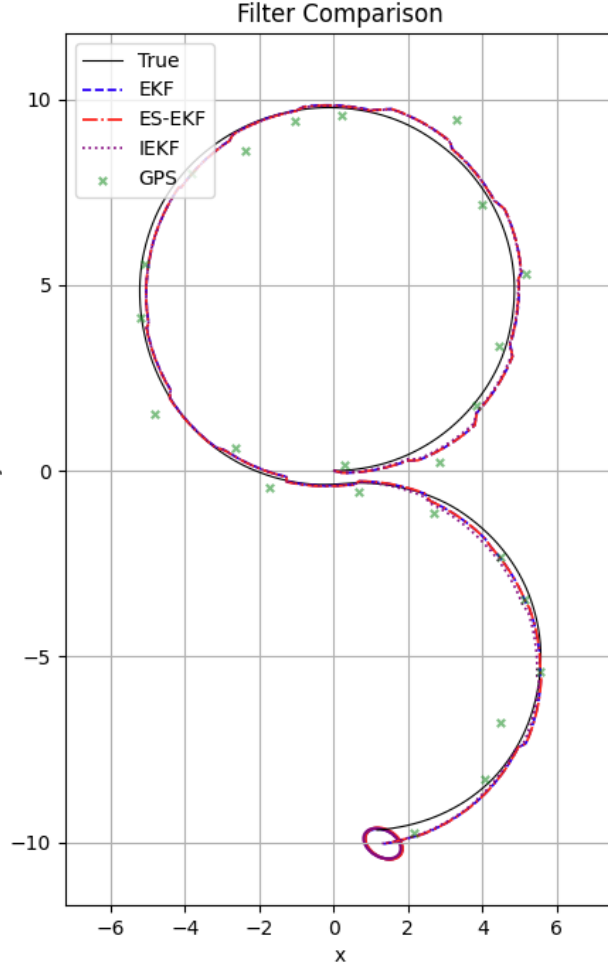


Figure 11: Covariance ellipses

4.5 Discussion.

The regular EKF has a covariance in the world frame so to plot it, we need to convert it to body frame which is corrupted by uncertainty in heading. The Error State EKF has covariance in the body frame already, so it is better than EKF. The IEKF also has the covariance ellipsoid described in body frame, so the ellipsoid should be similar to Error State EKF. However, the fact that in IEKF the error lives in lie algebra, might make covariance ellipsoid to be closer to true banana distribution than other ellipsoids.

The banana shape exists because rotating and translating do not commute on $SE(2)$. forward-velocity noise that accumulates while the vehicle turns produces a curved uncertainty cloud that cannot be captured by a straight ellipse in world coordinates. The world frame EKF rotates body noise into world coordinates at every step using current heading, so any heading error contaminates how forward velocity uncertainty is accumulated which produces nonlinearity. Working in the body frame keeps the process noise in its natural frame and delays the nonlinear coupling until the measurement update, so the filter no longer linearizes through the very nonlinearity that produces the banana shape.

The state independence property is the one that would be good to preserve. This makes the filter to be dependent only on control inputs and not the state, which eliminates the corruption of the estimation from the uncertainties in the state. This gives a consistency and the global convergence guarantees for the estimated state.

5 Unicycle Simulation in Modelica with Rumoca

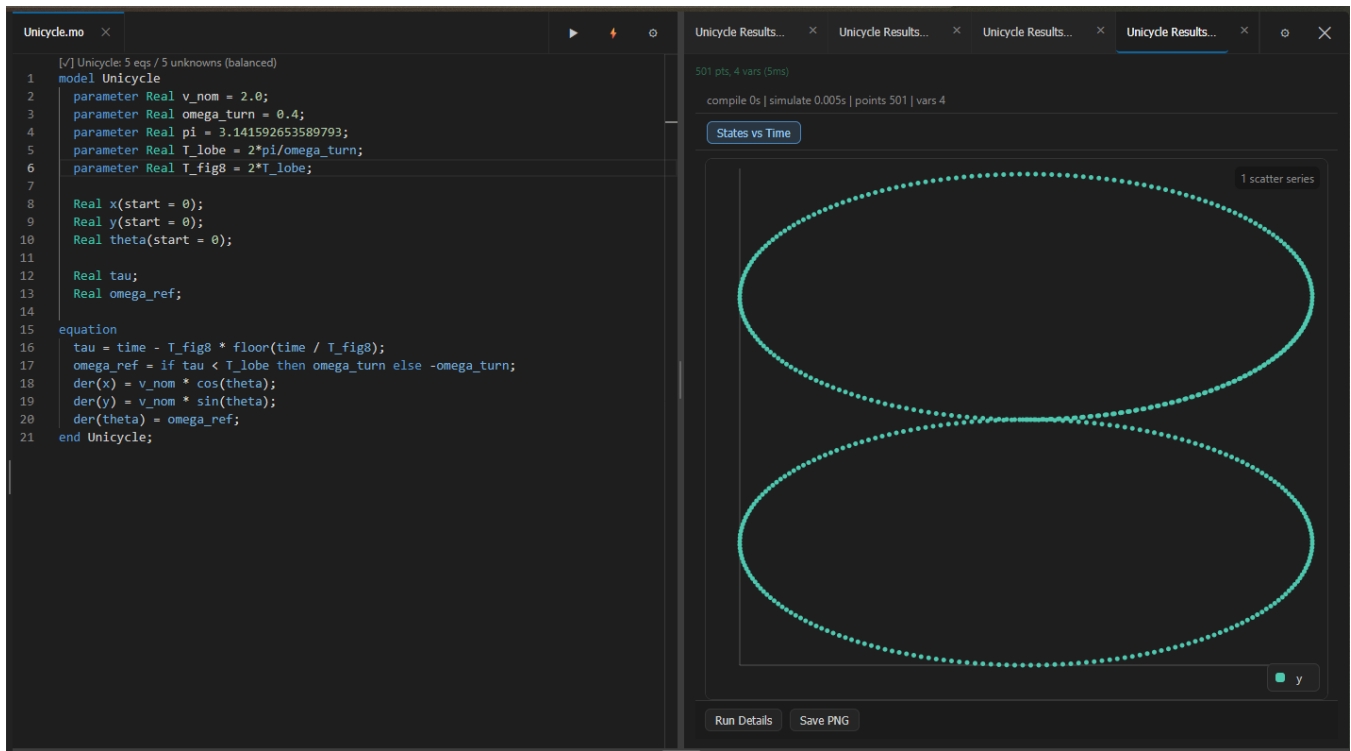


Figure 12: Modelica model

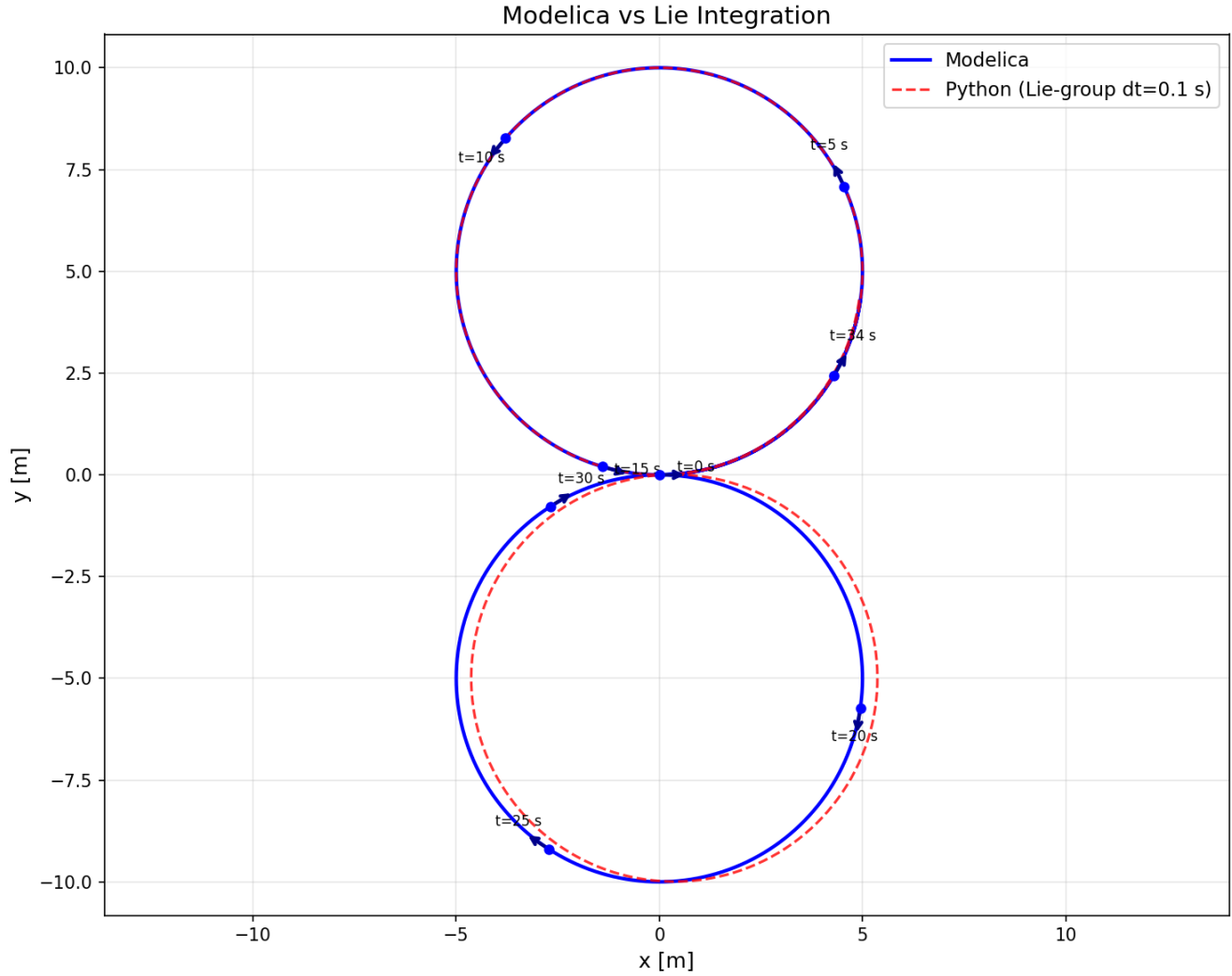


Figure 13: Unicycle Trajectory

The Modelica and Lie group trajectories are basically the same. However, on the lower lobe of the figure we see a difference. This is due to the time step of the Python method: the switch between negative and positive angular velocity does not happen instantaneously. In this case, the max difference is 7.361898×10^{-1} m. If we decrease the time step to 0.001 s, the trajectories match almost exactly and the max difference drops to 2.946335×10^{-4} m.

6 Appendix

Listing 1: Problem 1 code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import os, sys
4 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
5 from se2 import se2_exp, se2_log
6
7 pi = np.pi
8
9 v_nom = 2

```

```

10 w_turn = 0.4
11 r_fig = 5
12 sigma_v = 0.05
13 sigma_w = 0.5
14
15 T_lobe = 2 * pi / w_turn
16 T_fig8 = 10 * pi
17 T = 35
18
19 dt = 0.1
20
21 # Nominal trajectory
22 X0 = np.eye(3)
23 n_steps = int(T / dt)
24 X_ref = np.zeros((n_steps + 1, 3, 3))
25 X_ref[0] = X0
26 for i in range(n_steps):
27     t = i * dt
28     if t % T_fig8 < T_lobe:
29         w_ref = w_turn
30     else:
31         w_ref = -w_turn
32     xi = np.array([v_nom, 0, w_ref]) * dt
33     X_ref[i + 1] = X_ref[i] @ se2_exp(xi)
34
35 ## Monte Carlo simulation
36 N = 1000
37 snap_times = [5, 15, 30]
38 snap_steps = [int(ts / dt) for ts in snap_times]
39 X_snaps = {ts: np.zeros((N, 3, 3)) for ts in snap_times}
40 X_finals = np.zeros((N, 3, 3))
41
42 for j in range(N):
43     w_v = np.random.normal(0, sigma_v, n_steps)
44     w_w = np.random.normal(0, sigma_w, n_steps)
45     X = X0.copy()
46     for i in range(n_steps):
47         t = i * dt
48         if t % T_fig8 < T_lobe:
49             w_ref = w_turn
50         else:
51             w_ref = -w_turn
52         xi = np.array([v_nom + w_v[i], 0, w_ref + w_w[i]]) * dt
53         X = X @ se2_exp(xi)
54         for ts, ns in zip(snap_times, snap_steps):
55             if i + 1 == ns:
56                 X_snaps[ts][j] = X
57     X_finals[j] = X
58     if j % 100 == 0:
59         print(f"Iteration {j}/{N} completed")
60
61 # Extract final positions and headings
62 x_final = X_finals[:, 0, 2]
63 y_final = X_finals[:, 1, 2]
64 theta_final = np.arctan2(X_finals[:, 1, 0], X_finals[:, 0, 0])
65
66 # Statistics
67 positions = np.vstack([x_final, y_final])
68 p_bar = np.mean(positions, axis=1)

```

```

69 S = np.cov(positions)
70 print(f"Sample mean p_bar = [{p_bar[0]:.4f}, {p_bar[1]:.4f}]")
71 print(f"Sample covariance S =\n{S}")
72
73 # Covariance ellipse
74 t_ell = np.linspace(0, 2 * np.pi, 100)
75 circle = np.array([np.cos(t_ell), np.sin(t_ell)])
76 eigvals, eigvecs = np.linalg.eigh(S)
77 L = eigvecs @ np.diag(3 * np.sqrt(eigvals))
78 ellipse_pts = L @ circle + p_bar[:, None]
79
80 # Log-map errors:  $\text{eps}_i = \text{Log}(X_{\text{nom}}^{-1} @ X_i)^{\text{vee}}$   $R^3$ 
81 eps = np.array([se2_log(np.linalg.inv(X_ref[-1]) @ X_finals[i]) for i in range(N)])
82 eps_bar = np.mean(eps, axis=0)
83 Sigma = np.cov(eps.T)
84 print(f"Lie-algebra sample mean eps_bar = {eps_bar}")
85 print(f"Lie-algebra sample covariance Sigma =\n{Sigma}")
86
87 # 3 ellipsoid boundary in Lie algebra, projected to (x, y) via Exp map
88 phi_grid = np.linspace(0, np.pi, 50)
89 psi_grid = np.linspace(0, 2 * np.pi, 50)
90 PHI, PSI = np.meshgrid(phi_grid, psi_grid)
91 sphere = np.array([np.sin(PHI) * np.cos(PSI),
92                   np.sin(PHI) * np.sin(PSI),
93                   np.cos(PHI)]) # (3, 100, 50)
94
95 eigvals_e, eigvecs_e = np.linalg.eigh(Sigma)
96 VD_half = eigvecs_e @ np.diag(3 * np.sqrt(eigvals_e)) #  $V @ (3 * D^{\{1/2\}})$ 
97
98 X_nom = X_ref[-1]
99 ell_x = np.zeros_like(PHI)
100 ell_y = np.zeros_like(PHI)
101 for ii in range(PHI.shape[0]):
102     for jj in range(PHI.shape[1]):
103         eps_pt = eps_bar + VD_half @ sphere[:, ii, jj]
104         X_pt = X_nom @ se2_exp(eps_pt)
105         ell_x[ii, jj] = X_pt[0, 2]
106         ell_y[ii, jj] = X_pt[1, 2]
107
108 # --- Figure 1: final distribution at t = T ---
109 fig1, ax1 = plt.subplots()
110 ax1.plot(X_ref[:, 0, 2], X_ref[:, 1, 2], "b-", label="Nominal trajectory")
111 ax1.scatter(x_final, y_final, s=10, c="r", zorder=3, alpha=0.5, label="Final poses")
112 ax1.quiver(x_final, y_final, np.cos(theta_final), np.sin(theta_final),
113           scale=40, width=0.002, color="salmon", zorder=4, alpha=0.5)
114 ax1.plot(ellipse_pts[0], ellipse_pts[1], "g-", lw=2, label=r"Euclidean  $3\sigma$  ellipse")
115 ax1.scatter(ell_x, ell_y, s=5, c="m", alpha=0.3, zorder=5,
116           label=r"Exp( $3\sigma$  ellipsoid) projection")
117 ax1.set_xlabel("x [m]")
118 ax1.set_ylabel("y [m]")
119 ax1.axis("equal")
120 ax1.legend()
121 ax1.set_title("Monte Carlo Distribution at t = T")
122 fig1.tight_layout()
123 fig1.savefig(os.path.join(os.path.dirname(__file__), "banana_final.png"), dpi=150)
124
125 # --- Figure 2: intermediate snapshots at t = 5, 15, 30 ---
126 fig2, axes = plt.subplots(2, 2, figsize=(12, 10))

```

```

127 axes[1, 1].set_visible(False)
128 plot_axes = [axes[0, 0], axes[0, 1], axes[1, 0]]
129 for ax, ts in zip(plot_axes, snap_times):
130     Xs = X_snaps[ts]
131     xs = Xs[:, 0, 2]
132     ys = Xs[:, 1, 2]
133     ths = np.arctan2(Xs[:, 1, 0], Xs[:, 0, 0])
134     ref_idx = int(ts / dt)
135     ax.plot(X_ref[:ref_idx + 1, 0, 2], X_ref[:ref_idx + 1, 1, 2], "b-",
136             label="Nominal trajectory")
137     ax.scatter(xs, ys, s=10, c="r", zorder=3, alpha=0.5, label="Poses")
138     ax.quiver(xs, ys, np.cos(ths), np.sin(ths),
139              scale=40, width=0.002, color="salmon", zorder=4, alpha=0.5)
140     ax.set_xlabel("x [m]")
141     ax.set_ylabel("y [m]")
142     ax.axis("equal")
143     ax.legend(fontsize=8)
144     ax.set_title(f"t = {ts} s")
145 fig2.suptitle("Intermediate Snapshots", fontsize=14)
146 fig2.tight_layout()
147 fig2.savefig(os.path.join(os.path.dirname(__file__), "banana_snapshots.png"), dpi=150)
148
149 # --- Figure 3: Lie-algebra error projections (eps1-eps2 and eps1-eps3) ---
150 t_ell = np.linspace(0, 2 * np.pi, 200)
151 circle = np.array([np.cos(t_ell), np.sin(t_ell)])
152 labels = [r"$\varepsilon_1$", r"$\varepsilon_2$", r"$\varepsilon_3$"]
153 proj_pairs = [(0, 1), (0, 2)]
154
155 fig3, axes3 = plt.subplots(1, 2, figsize=(12, 5))
156 for ax, (i, j) in zip(axes3, proj_pairs):
157     ax.scatter(eps[:, i], eps[:, j], s=5, c="r", alpha=0.5)
158     sub_mean = eps_bar[[i, j]]
159     sub_cov = Sigma[np.ix_([i, j], [i, j])]
160     eigvals_s, eigvecs_s = np.linalg.eigh(sub_cov)
161     L_s = eigvecs_s @ np.diag(3 * np.sqrt(eigvals_s))
162     ell = L_s @ circle + sub_mean[:, None]
163     ax.plot(ell[0], ell[1], "g-", lw=2, label=r"$3\sigma$ ellipse")
164     ax.plot(sub_mean[0], sub_mean[1], "g+", ms=12, mew=2)
165     ax.set_xlabel(labels[i])
166     ax.set_ylabel(labels[j])
167     ax.axis("equal")
168     ax.legend()
169     ax.set_title(f"{labels[i]} vs {labels[j]}")
170 fig3.suptitle("Lie-Algebra Error Distribution", fontsize=14)
171 fig3.tight_layout()
172 fig3.savefig(os.path.join(os.path.dirname(__file__), "banana_lie_errors.png"), dpi=150)
173
174 plt.show()

```

Listing 2: Unicycle modelica code

```

1 model Unicycle
2   parameter Real v_nom = 2.0;
3   parameter Real omega_turn = 0.4;
4   parameter Real pi = 3.141592653589793;
5   parameter Real T_lobe = 2*pi/omega_turn;
6   parameter Real T_fig8 = 2*T_lobe;
7
8   Real x(start = 0);

```

```

9   Real y(start = 0);
10  Real theta(start = 0);
11
12  Real tau;
13  Real omega_ref;
14
15  equation
16    tau = time - T_fig8 * floor(time / T_fig8);
17    omega_ref = if tau < T_lobe then omega_turn else -omega_turn;
18    der(x) = v_nom * cos(theta);
19    der(y) = v_nom * sin(theta);
20    der(theta) = omega_ref;
21  end Unicycle;

```

Listing 3: Rumoca generated code

```

1   """
2   Generated by Rumoca.
3
4   This template renders the current DAE context into a SymPy-friendly symbolic
5   model. It keeps Rumoca's continuous equations in residual form and can solve
6   for explicit state derivatives and algebraics when possible.
7   """
8
9
10  import sympy as sp
11
12
13  def der(symbol):
14      if isinstance(symbol, sp.Indexed):
15          return sp.IndexedBase(f"{symbol.base}_dot")[symbol.indices]
16      if isinstance(symbol, sp.IndexedBase):
17          return sp.IndexedBase(f"{symbol}_dot")
18      return sp.Symbol(f"{symbol}_dot")
19
20
21  def pre(symbol):
22      if isinstance(symbol, sp.Indexed):
23          return sp.IndexedBase(f"pre_{symbol.base}")[symbol.indices]
24      if isinstance(symbol, sp.IndexedBase):
25          return sp.IndexedBase(f"pre_{symbol}")
26      return sp.Symbol(f"pre_{symbol}")
27
28
29  def if_else(condition, then_expr, else_expr):
30      return sp.Piecewise((then_expr, condition), (else_expr, True))
31
32
33  def _vector(entries):
34      return (
35          sp.Matrix([symbol for _, symbol, _ in entries]),
36          {name: start for name, _, start in entries},
37          {name: index for index, (name, _, _) in enumerate(entries)},
38      )
39
40
41  def _column(entries):
42      return sp.Matrix(entries) if entries else sp.Matrix([])
43

```

```

44
45 def zeros(*dims):
46     if len(dims) == 1:
47         return sp.Matrix.zeros(int(dims[0]), 1)
48     return sp.Matrix.zeros(*(int(dim) for dim in dims))
49
50
51 def ones(*dims):
52     if len(dims) == 1:
53         return sp.Matrix.ones(int(dims[0]), 1)
54     return sp.Matrix.ones(*(int(dim) for dim in dims))
55
56
57 def sum(values):
58     if isinstance(values, sp.MatrixBase):
59         return sp.Add(*list(values))
60     if isinstance(values, (list, tuple)):
61         return sp.Add(*values)
62     return values
63
64
65 abs = sp.Abs
66 min = sp.Min
67 max = sp.Max
68 sign = sp.sign
69 sqrt = sp.sqrt
70 sin = sp.sin
71 cos = sp.cos
72 tan = sp.tan
73 asin = sp.asin
74 acos = sp.acos
75 atan = sp.atan
76 atan2 = sp.atan2
77 sinh = sp.sinh
78 cosh = sp.cosh
79 tanh = sp.tanh
80 exp = sp.exp
81 log = sp.log
82 log10 = lambda value: sp.log(value, 10)
83 floor = sp.floor
84 ceil = sp.ceiling
85 transpose = lambda value: sp.Matrix(value).T
86 identity = sp.eye
87 cross = lambda lhs, rhs: sp.Matrix(lhs).cross(sp.Matrix(rhs))
88 div = lambda lhs, rhs: sp.floor(lhs / rhs)
89 mod = sp.Mod
90 rem = sp.Mod
91
92
93 class Model:
94     """Flattened symbolic DAE model."""
95
96     def __init__(self):
97         self.time = sp.Symbol("time")
98         time = self.time
99
100         # States, algebraics, inputs, outputs, parameters, constants, discretetes.
101
102

```

```

103     theta = sp.Symbol("theta")
104
105
106     x = sp.Symbol("x")
107
108
109     y = sp.Symbol("y")
110
111
112     self.x, self.x_start, self.x_index = _vector([
113         ("theta", theta, 0),
114         ("x", x, 0),
115         ("y", y, 0)
116     ])
117
118
119
120     omega_ref = sp.Symbol("omega_ref")
121
122
123     tau = sp.Symbol("tau")
124
125
126     self.y, self.y_start, self.y_index = _vector([
127         ("omega_ref", omega_ref, 0),
128         ("tau", tau, 0)
129     ])
130
131
132     self.u, self.u_start, self.u_index = _vector([])
133
134
135     self.w, self.w_start, self.w_index = _vector([])
136
137
138
139     T_fig8 = sp.Symbol("T_fig8")
140
141
142     T_lobe = sp.Symbol("T_lobe")
143
144
145     omega_turn = sp.Symbol("omega_turn")
146
147
148     pi = sp.Symbol("pi")
149
150
151     v_nom = sp.Symbol("v_nom")
152
153
154     self.p, self.p_start, self.p_index = _vector([
155         ("T_fig8", T_fig8, (2 * T_lobe)),
156         ("T_lobe", T_lobe, ((2 * pi) / omega_turn)),
157         ("omega_turn", omega_turn, 0.4),
158         ("pi", pi, 3.141592653589793),
159         ("v_nom", v_nom, 2)
160     ])
161

```



```

162     self.constants, self.constants_start, self.constants_index = _vector([])
163
164
165
166     self.z, self.z_start, self.z_index = _vector([])
167
168
169
170     self.m, self.m_start, self.m_index = _vector([])
171
172
173
174     self.x_dot_alias, self.x_dot_alias_start, self.x_dot_alias_index = _vector([])
175
176     self.f_x = _column([
177         (tau - (time - (T_fig8 * floor((time / T_fig8))))),
178         (omega_ref - if_else((tau < T_lobe), omega_turn, (-omega_turn))),
179         (der(x) - (v_nom * cos(theta))),
180         (der(y) - (v_nom * sin(theta))),
181         (der(theta) - omega_ref)
182     ])
183
184     self.initial_residuals = _column([])
185
186     self.relations = _column([
187         (tau < T_lobe)
188     ])
189
190     self.explicit_targets = [
191         tau,
192
193         omega_ref,
194
195         der(x),
196
197         der(y),
198
199         der(theta),
200
201     ]
202
203     self.explicit_solution = None
204     self.explicit_rhs = sp.Matrix([])
205
206
207
208     def solve_explicit(self):
209         """Best-effort symbolic solve for explicit derivatives and algebraics."""
210         if len(self.f_x) == 0 or len(self.explicit_targets) == 0:
211             self.explicit_solution = {}
212             self.explicit_rhs = sp.Matrix([])
213             return self.explicit_solution
214
215         solutions = sp.solve(
216             list(self.f_x),
217             self.explicit_targets,
218             dict=True,
219             simplify=False,
220         )

```

```

221     self.explicit_solution = solutions[0] if solutions else {}
222     self.explicit_rhs = sp.Matrix(
223         [
224             self.explicit_solution.get(symbol, symbol)
225             for symbol in self.explicit_targets
226         ]
227     )
228     return self.explicit_solution
229
230 def summary(self):
231     return {
232         "states": [str(symbol) for symbol in self.x],
233         "algebraics": [str(symbol) for symbol in self.y],
234         "inputs": [str(symbol) for symbol in self.u],
235         "parameters": [str(symbol) for symbol in self.p],
236         "continuous_residual_count": len(self.f_x),
237         "initial_residual_count": len(self.initial_residuals),
238         "explicit_target_count": len(self.explicit_targets),
239     }
240
241
242 if __name__ == "__main__":
243     model = Model()
244     print(model.summary())
245     if len(model.f_x) > 0 and len(model.explicit_targets) > 0:
246         print(model.solve_explicit())

```

Listing 4: Unicycle sim code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp
4 from scipy.interpolate import interp1d
5 import sympy as sp
6 import sys, os
7
8 HERE = os.path.dirname(os.path.abspath(__file__))
9 sys.path.insert(0, os.path.join(HERE, ".."))
10 from se2 import se2_exp
11
12 sys.path.insert(0, HERE)
13 from rumoca import Model
14
15 model = Model()
16
17 omega_turn_val = 0.4
18 v_nom_val = 2.0
19 T_lobe_val = 2 * np.pi / omega_turn_val
20 T_fig8_val = 2 * T_lobe_val
21
22
23 def ode_rhs(t, state):
24     theta = state[0]
25     tau = t - T_fig8_val * np.floor(t / T_fig8_val)
26     omega = omega_turn_val if tau < T_lobe_val else -omega_turn_val
27     return [omega, v_nom_val * np.cos(theta), v_nom_val * np.sin(theta)]
28
29
30 T = 35.0

```

```

31 x0 = [0.0, 0.0, 0.0] # [theta, x, y]
32 t_eval = np.linspace(0, T, 3501)
33
34 sol = solve_ivp(
35     ode_rhs, [0, T], x0,
36     method="RK45",
37     t_eval=t_eval,
38     rtol=1e-10, atol=1e-12,
39     max_step=0.01,
40 )
41
42 t_mod = sol.t
43 theta_mod = sol.y[0]
44 x_mod = sol.y[1]
45 y_mod = sol.y[2]
46
47
48 # Lie group
49 dt_py = 0.001
50 n_steps = int(T / dt_py)
51 X_ref = np.zeros((n_steps + 1, 3, 3))
52 X_ref[0] = np.eye(3)
53
54 for i in range(n_steps):
55     t = i * dt_py
56     tau = t % T_lobe_val
57     w_ref = omega_turn_val if tau < T_lobe_val else -omega_turn_val
58     xi = np.array([v_nom_val, 0, w_ref]) * dt_py
59     X_ref[i + 1] = X_ref[i] @ se2_exp(xi)
60
61 t_py = np.arange(n_steps + 1) * dt_py
62 x_py = X_ref[:, 0, 2]
63 y_py = X_ref[:, 1, 2]
64 theta_py = np.arctan2(X_ref[:, 1, 0], X_ref[:, 0, 0])
65
66 x_mod_at_py = interp1d(t_mod, x_mod, kind="cubic")(t_py)
67 y_mod_at_py = interp1d(t_mod, y_mod, kind="cubic")(t_py)
68
69 pos_err = np.sqrt((x_mod_at_py - x_py)**2 + (y_mod_at_py - y_py)**2)
70 max_pos_err = np.max(pos_err)
71 idx_max = np.argmax(pos_err)
72
73 print(f" max_k ||p_Modelica - p_Python|| = {max_pos_err:.6e} m")
74 print(f" at t = {t_py[idx_max]:.1f} s")
75
76
77 fig, ax = plt.subplots(figsize=(10, 8))
78
79 ax.plot(x_mod, y_mod, "b-", lw=2, label="Modelica")
80 ax.plot(x_py, y_py, "r--", lw=1.5, alpha=0.8,
81         label=f"Python (Lie-group dt={dt_py} s)")
82
83 arrow_times = [0, 5, 10, 15, 20, 25, 30, 34]
84 for ta in arrow_times:
85     idx = np.argmin(np.abs(t_mod - ta))
86     cx, cy = x_mod[idx], y_mod[idx]
87     dx = np.cos(theta_mod[idx])
88     dy = np.sin(theta_mod[idx])
89     ax.annotate(

```

```

90         "", xy=(cx + 0.7 * dx, cy + 0.7 * dy), xytext=(cx, cy),
91         arrowprops=dict(arrowstyle="->", color="darkblue", lw=2),
92     )
93     ax.plot(cx, cy, "bo", ms=5, zorder=5)
94     ax.text(cx + 0.9 * dx, cy + 0.9 * dy, f"t={ta} s",
95            fontsize=8, ha="center", va="bottom")
96
97     ax.set_xlabel("x [m]", fontsize=12)
98     ax.set_ylabel("y [m]", fontsize=12)
99     ax.set_title("Modelica vs Lie Integration",
100                fontsize=14)
101     ax.legend(fontsize=11, loc="best")
102     ax.axis("equal")
103     ax.grid(True, alpha=0.3)
104     fig.tight_layout()
105     out_path = os.path.join(HERE, "unicycle_trajectory.png")
106     fig.savefig(out_path, dpi=150)
107     print(f"\nFigure saved to {out_path}")
108     plt.show()

```

```
In [16]: import sympy
import numpy as np
import casadi as ca
import matplotlib.pyplot as plt
from scipy.stats import chi2
from matplotlib.patches import Ellipse
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
import sys, os
sys.path.append(os.path.abspath('.'))
from se2 import se2_exp, se2_Ad
```

```
In [17]: def ES_EKF():
    # Initialize variables
    x_nom = ca.SX.sym("x_nom", 3)
    P = ca.SX.sym("P", 3, 3)
    v = ca.SX.sym("v")
    w = ca.SX.sym("w")
    dt = ca.SX.sym("dt")
    sigma_v = ca.SX.sym("sigma_v")
    sigma_w = ca.SX.sym("sigma_w")
    sigma_gps = ca.SX.sym("sigma_gps")
    z_gps = ca.SX.sym("z_gps", 2)
    gps_flag = ca.SX.sym("gps_flag")

    A_error = ca.vertcat(
        ca.horzcat(1, w*dt, 0),
        ca.horzcat(-w*dt, 1, v*dt),
        ca.horzcat(0, 0, 1)
    )
    H = ca.vertcat(
        ca.horzcat(ca.cos(x_nom[2]), -ca.sin(x_nom[2]), 0),
        ca.horzcat(ca.sin(x_nom[2]), ca.cos(x_nom[2]), 0)
    )
    Q_body = ca.diag(ca.vertcat(sigma_v**2, 0, sigma_w**2))
    Q_gps = ca.diag(ca.vertcat(sigma_gps**2, sigma_gps**2))

    # Propagate state
    x_prop = ca.vertcat(x_nom[0] + v*ca.cos(x_nom[2])*dt,
                        x_nom[1] + v*ca.sin(x_nom[2])*dt,
                        x_nom[2] + w*dt)

    # Prediction
    P_pred = A_error @ P @ A_error.T + Q_body*dt**2

    p = ca.vertcat(x_prop[0], x_prop[1])

    # Update
    y = z_gps - p
    S = H @ P_pred @ H.T + Q_gps
    K = P_pred @ H.T @ ca.inv(S)
    P_meas = (ca.SX.eye(3) - K @ H) @ P_pred @ (ca.SX.eye(3) - K @ H).T + K @ Q_gps
    de = K @ y
```

```

x_meas = ca.vertcat(x_prop[0] + ca.cos(x_prop[2])*de[0] - ca.sin(x_prop[2])*de[
                    x_prop[1] + ca.sin(x_prop[2])*de[0] + ca.cos(x_prop[2])*de[
                    x_prop[2] + de[2])

x_new = ca.if_else(gps_flag, x_meas, x_prop)
P_new = ca.if_else(gps_flag, P_meas, P_pred)
nu = ca.if_else(gps_flag, y.T @ ca.inv(S) @ y, np.nan)

f_filter = ca.Function(
    "error_state_filter",
    [
        x_nom,
        P,
        v,
        w,
        dt,
        sigma_v,
        sigma_w,
        sigma_gps,
        z_gps,
        gps_flag
    ],
    [x_new, P_new, nu],
    [
        "x_nom",
        "P",
        "v",
        "w",
        "dt",
        "sigma_v",
        "sigma_w",
        "sigma_gps",
        "z_gps",
        "gps_flag"
    ],
    ["x_new", "P_new", "nu"],
)
return {"error_state_filter": f_filter}

```

In [18]: `def IEKF(X, P, v, w, dt, sigma_v, sigma_w, sigma_gps, z_gps, gps_flag):`

```

xi = np.array([v, 0, w])
theta = np.arctan2(X[1, 0], X[0, 0])

M = se2_exp(-np.array([v, 0, w]) * dt)  # 3x3 SE(2) matrix
pose = np.array([M[0, 2],
                  M[1, 2],
                  np.arctan2(M[1, 0], M[0, 0])])
A_k = se2_Ad(pose)

H = np.array([
    [np.cos(theta), -np.sin(theta), 0],
    [np.sin(theta),  np.cos(theta), 0]
])
Q_body = np.diag(np.array([sigma_v**2, 0, sigma_w**2]))

```

```

Q_gps = np.diag(np.array([sigma_gps**2, sigma_gps**2]))

# Propagate state
X = X @ se2_exp(xi * dt)

# Prediction
P = A_k @ P @ A_k.T + Q_body*dt**2

if not gps_flag:
    return X, P, np.nan

# Update
p = np.array([X[0, 2], X[1, 2]])
S = H @ P @ H.T + Q_gps
K = P @ H.T @ np.linalg.inv(S)
P = (np.eye(3) - K @ H) @ P @ (np.eye(3) - K @ H).T + K @ Q_gps @ K.T
y = z_gps - p
nu = y @ np.linalg.inv(S) @ y
de = K @ y

X = X @ se2_exp(de)

return X, P, nu

```

```

In [19]: class Model():
def __init__(self):
    self.x = sympy.symbols('x')
    self.y = sympy.symbols('y')
    self.theta = sympy.symbols('theta')
    self.dt = sympy.symbols('dt')
    self.v_nom = sympy.symbols('v_nom')
    self.w_ref = sympy.symbols('w_ref')
    self.P = sympy.MatrixSymbol('P', 3, 3)

def dynamics(self):
    x_next = self.x + self.v_nom * sympy.cos(self.theta)*self.dt
    y_next = self.y + self.v_nom * sympy.sin(self.theta)*self.dt
    theta_next = self.theta + self.w_ref * self.dt
    return sympy.Matrix([x_next, y_next, theta_next])

def state(self):
    return sympy.Matrix([self.x, self.y, self.theta])

def Jacobian(self):
    return self.dynamics().jacobian(self.state())

def Q_body(self):
    sigma_v, sigma_w = sympy.symbols('sigma_v sigma_w')
    return sympy.diag(sigma_v**2, 0, sigma_w**2)

def Q(self):
    Lk = self.dt * sympy.Matrix([
        [sympy.cos(self.theta), 0, 0],
        [sympy.sin(self.theta), 0, 0],
        [0, 0, 1]
    ])

```

```

        return Lk * self.Q_body() * Lk.T

    def measurement_noise(self):
        sigma_gps = sympy.symbols('sigma_gps')
        R = sigma_gps**2 * sympy.eye(2)
        return R

    def measurement_jacobian(self):
        return sympy.Matrix([self.x, self.y]).jacobian(self.state())

```

```

In [20]: class EKF():
    def __init__(self, model):
        syms = [model.x, model.y, model.theta, model.v_nom, model.w_ref, model.dt]
        noise_syms = sympy.symbols('sigma_v sigma_w sigma_gps')

        self.f = sympy.lambdify(syms, model.dynamics(), 'numpy')
        self.F = sympy.lambdify(syms, model.Jacobian(), 'numpy')
        self.Q = sympy.lambdify([model.theta, model.dt, noise_syms[0], noise_syms[1],
                                noise_syms[2], noise_syms[3], noise_syms[4], noise_syms[5]],
                                model.Q_body(), 'numpy')
        self.H = np.array(model.measurement_jacobian()).astype(float)
        self.R = sympy.lambdify([noise_syms[2]], model.measurement_noise(), 'numpy')

    def predict(self, x_est, P_est, v, w, dt, sigma_v, sigma_w):
        x_pred = np.array(self.f(*x_est, v, w, dt)).flatten()
        F_val = np.array(self.F(*x_est, v, w, dt)).astype(float)
        Q_val = np.array(self.Q(x_est[2], dt, sigma_v, sigma_w)).astype(float)
        P_pred = F_val @ P_est @ F_val.T + Q_val
        return x_pred, P_pred

    def update(self, x_pred, P_pred, z, sigma_gps):
        H = self.H
        R = np.array(self.R(sigma_gps)).astype(float)
        K = P_pred @ H.T @ np.linalg.inv(H @ P_pred @ H.T + R)
        x_est = x_pred + K @ (z - H @ x_pred)
        P_est = (np.eye(3) - K @ H) @ P_pred @ (np.eye(3) - K @ H).T + K @ R @ K.T
        return x_est, P_est

    def simulate_step(self, x_est, P_est, z, v, w, dt, sigma_v, sigma_w, sigma_gps):
        x_pred, P_pred = self.predict(x_est, P_est, v, w, dt, sigma_v, sigma_w)
        x_est, P_est = self.update(x_pred, P_pred, z, sigma_gps)
        return x_est, P_est

```

```

In [21]: # Model
model = Model()
model.dynamics()
kf = EKF(model)
rng = np.random.default_rng(67676767)

```

```

In [22]: # Constants
pi = np.pi

# Parameters
v_nom = 2
w_turn = 0.4
r_fig = 5

```



```

sigma_v = 1.0
sigma_w = 0.05
sigma_gps = 0.3
dt = 0.1

# Simulation parameters
T_lobe = 2 * pi / w_turn
T_fig8 = 10 * pi
T = 35
n_steps = int(T / dt)
gps_freq = 1

# Initial conditions
x0_true = np.array([0.0, 0.0, 0.0])
x0_est = np.array([0.0, 0.0, 0.0])
P0 = np.diag([0.01, 0.01, 0.01])
H = kf.H
R = np.array(kf.R(sigma_gps)).astype(float)

# Pre-compute reference inputs and GPS schedule
w_ref_vec = np.zeros(n_steps)
gps_flags = np.zeros(n_steps, dtype=bool)
for i in range(n_steps):
    t = i * dt
    w_ref_vec[i] = w_turn if t % T_fig8 < T_lobe else -w_turn
    gps_flags[i] = t % (1 / gps_freq) < 0.01

# Pre-generate noise so all filters use the same realization
noise_v = sigma_v * rng.standard_normal(n_steps)
noise_w = sigma_w * rng.standard_normal(n_steps)
noise_gps = rng.multivariate_normal(np.zeros(2), R, size=n_steps)

# Generate true trajectory and GPS measurements (shared by all filters)
x_true_hist = np.zeros((n_steps + 1, 3))
x_true_hist[0] = x0_true
z_hist = np.full((n_steps, 2), np.nan)

X_true = np.eye(3)
for i in range(n_steps):
    xi = np.array([v_nom + noise_v[i], 0, w_ref_vec[i] + noise_w[i]]) * dt
    X_true = X_true @ se2_exp(xi)
    x_true_hist[i+1] = [X_true[0, 2], X_true[1, 2], np.arctan2(X_true[1, 0], X_true[0, 0])]
    if gps_flags[i]:
        z_hist[i] = H @ x_true_hist[i+1] + noise_gps[i]

```

```

In [23]: x_est_hist = np.zeros((n_steps + 1, 3))
P_est_hist = np.zeros((n_steps + 1, 3, 3))
x_est_hist[0] = x0_est
P_est_hist[0] = P0

x_est = x0_est.copy()
P_est = P0.copy()
for i in range(n_steps):
    x_est, P_est = kf.predict(x_est, P_est, v_nom, w_ref_vec[i], dt, sigma_v, sigma_w)
    if gps_flags[i]:
        x_est, P_est = kf.update(x_est, P_est, z_hist[i], sigma_gps)

```

```

x_est_hist[i + 1] = x_est
P_est_hist[i + 1] = P_est

```

```

In [24]: x_est_hist_es = np.zeros((n_steps + 1, 3))
P_est_hist_es = np.zeros((n_steps + 1, 3, 3))
x_est_hist_es[0] = x0_est
P_est_hist_es[0] = P0

fns = ES_EKF()
es_ekf_fn = fns["error_state_filter"]

x_est = x0_est.copy()
P_est = P0.copy()
for i in range(n_steps):
    gps_flag = 1 if gps_flags[i] else 0
    z_gps = z_hist[i] if gps_flags[i] else np.zeros(2)
    x_est, P_est, _ = es_ekf_fn(
        x_est, P_est, v_nom, w_ref_vec[i], dt,
        sigma_v, sigma_w, sigma_gps, z_gps, gps_flag)
    x_est_hist_es[i + 1] = np.array(x_est).flatten()
    P_est_hist_es[i + 1] = np.array(P_est)

```

```

In [25]: x_est_hist_iekf = np.zeros((n_steps + 1, 3))
P_est_hist_iekf = np.zeros((n_steps + 1, 3, 3))
x_est_hist_iekf[0] = x0_est
P_est_hist_iekf[0] = P0

X_est = np.eye(3)
P_est = P0.copy()
for i in range(n_steps):
    gps_flag = 1 if gps_flags[i] else 0
    z_gps = z_hist[i] if gps_flags[i] else np.zeros(2)
    X_est, P_est, _ = IEKF(
        X_est, P_est, v_nom, w_ref_vec[i], dt,
        sigma_v, sigma_w, sigma_gps, z_gps, gps_flag)
    x_est_hist_iekf[i + 1] = [X_est[0, 2], X_est[1, 2], np.arctan2(X_est[1, 0], X_e
    P_est_hist_iekf[i + 1] = P_est

```

```

In [26]: def cov_ellipse(P_2x2, center, n_std=3.0, **kwargs):
    eigvals, eigvecs = np.linalg.eigh(P_2x2)
    order = eigvals.argsort()[::-1]
    eigvals = eigvals[order]
    eigvecs = eigvecs[:, order]
    angle = np.degrees(np.arctan2(eigvecs[1, 0], eigvecs[0, 0]))
    w, h = 2 * n_std * np.sqrt(eigvals)
    return Ellipse(xy=center, width=w, height=h, angle=angle,
        fill=False, linewidth=1.5, **kwargs)

def body_to_world_cov(P, theta):
    R_th = np.array([[np.cos(theta), -np.sin(theta), 0],
        [np.sin(theta), np.cos(theta), 0],
        [0, 0, 1]])
    return R_th @ P @ R_th.T

fig, ax = plt.subplots(figsize=(8, 8))

```

```

ax.set_aspect('equal')
ax.grid(True)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Filter Comparison')

true_line, = ax.plot([], [], 'k-', linewidth=0.8, label='True')
ekf_line, = ax.plot([], [], 'b--', linewidth=1.2, label='EKF')
es_line, = ax.plot([], [], 'r-.', linewidth=1.2, label='ES-EKF')
iekf_line, = ax.plot([], [], color='purple', linestyle=':', linewidth=1.2, label='I
gps_scatter = ax.scatter([], [], c='green', s=15, marker='x', label='GPS', zorder=5
ellipse_patch_ekf = None
ellipse_patch_es = None
ellipse_patch_iekf = None
ax.legend(loc='upper left')

pad = 2
ax.set_xlim(x_true_hist[:, 0].min() - pad, x_true_hist[:, 0].max() + pad)
ax.set_ylim(x_true_hist[:, 1].min() - pad, x_true_hist[:, 1].max() + pad)

skip = 5

def init():
    true_line.set_data([], [])
    ekf_line.set_data([], [])
    es_line.set_data([], [])
    iekf_line.set_data([], [])
    gps_scatter.set_offsets(np.empty((0, 2)))
    return true_line, ekf_line, es_line, iekf_line, gps_scatter

def animate(frame):
    global ellipse_patch_ekf, ellipse_patch_es, ellipse_patch_iekf
    k = frame * skip

    true_line.set_data(x_true_hist[:k+1, 0], x_true_hist[:k+1, 1])
    ekf_line.set_data(x_est_hist[:k+1, 0], x_est_hist[:k+1, 1])
    es_line.set_data(x_est_hist_es[:k+1, 0], x_est_hist_es[:k+1, 1])
    iekf_line.set_data(x_est_hist_iekf[:k+1, 0], x_est_hist_iekf[:k+1, 1])

    z_slice = z_hist[:k]
    gps_mask = ~np.isnan(z_slice[:, 0])
    if gps_mask.any():
        gps_scatter.set_offsets(z_slice[gps_mask])

    if ellipse_patch_ekf is not None:
        ellipse_patch_ekf.remove()
    ellipse_patch_ekf = cov_ellipse(P_est_hist[k, :2, :2], x_est_hist[k, :2], edgeco
    ax.add_patch(ellipse_patch_ekf)

    if ellipse_patch_es is not None:
        ellipse_patch_es.remove()
    P_world_es = body_to_world_cov(P_est_hist_es[k], x_est_hist_es[k, 2])
    ellipse_patch_es = cov_ellipse(P_world_es[:2, :2], x_est_hist_es[k, :2], edgeco
    ax.add_patch(ellipse_patch_es)

    if ellipse_patch_iekf is not None:

```

```

        ellipse_patch_iekf.remove()
        P_world_iekf = body_to_world_cov(P_est_hist_iekf[k], x_est_hist_iekf[k, 2])
        ellipse_patch_iekf = cov_ellipse(P_world_iekf[:2, :2], x_est_hist_iekf[k, :2],
        ax.add_patch(ellipse_patch_iekf)

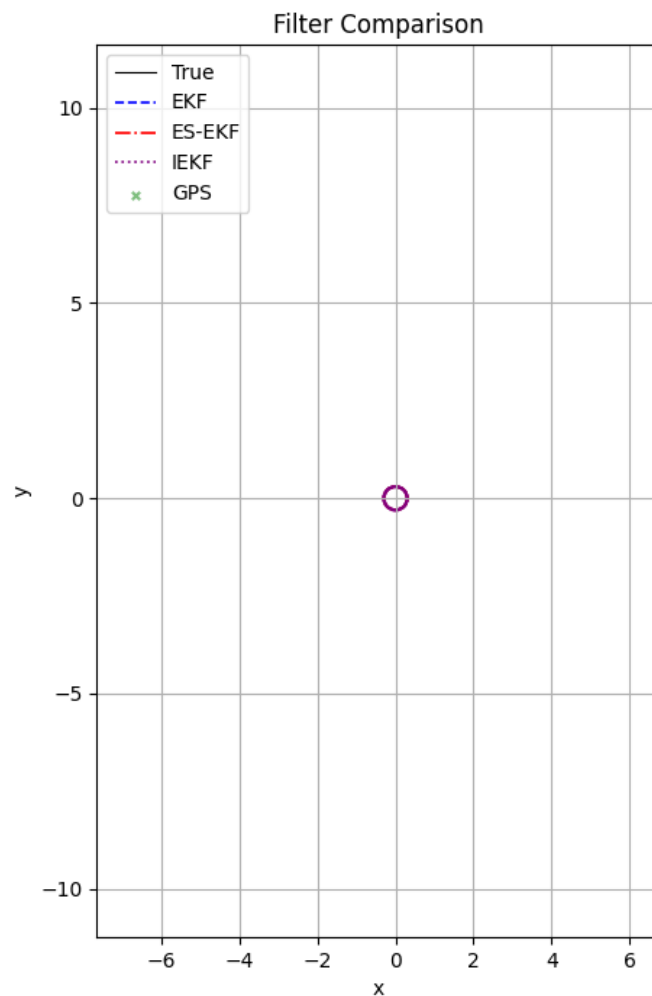
    return true_line, ekf_line, es_line, iekf_line, gps_scatter, ellipse_patch_ekf,

n_frames = (n_steps + 1) // skip
anim = FuncAnimation(fig, animate, init_func=init, frames=n_frames, interval=30, bl
anim.save('filter_comparison.gif', writer='pillow', fps=30)
plt.close(fig)
print('Saved filter_comparison.gif')
HTML(anim.to_jshtml())

```

Saved filter_comparison.gif

Out[26]:



☐ Once
 ☒ Loop
 ☐ Reflect

```

In [27]: N = 100
mc_rng = np.random.default_rng(12345)

epsilon_all_ekf = np.zeros((N, n_steps))
nu_all_ekf      = np.full((N, n_steps), np.nan)
epsilon_all_es  = np.zeros((N, n_steps))
nu_all_es       = np.full((N, n_steps), np.nan)
epsilon_all_iekf = np.zeros((N, n_steps))
nu_all_iekf     = np.full((N, n_steps), np.nan)

fns = ES_EKF()
es_ekf_fn = fns["error_state_filter"]

for j in range(N):
    mc_nv = sigma_v * mc_rng.standard_normal(n_steps)
    mc_nw = sigma_w * mc_rng.standard_normal(n_steps)
    mc_gps = mc_rng.multivariate_normal(np.zeros(2), R, size=n_steps)

    X_true_mc = np.eye(3)
    x_true_mc = np.zeros((n_steps + 1, 3))
    x_true_mc[0] = x0_true
    z_mc = np.full((n_steps, 2), np.nan)

    for i in range(n_steps):
        xi = np.array([v_nom + mc_nv[i], 0, w_ref_vec[i] + mc_nw[i]]) * dt
        X_true_mc = X_true_mc @ se2_exp(xi)
        x_true_mc[i+1] = [X_true_mc[0,2], X_true_mc[1,2], np.arctan2(X_true_mc[1,0], X_true_mc[0,0])]
        if gps_flags[i]:
            z_mc[i] = H @ x_true_mc[i+1] + mc_gps[i]

    # --- EKF ---
    x_est = x0_est.copy(); P_est = P0.copy()
    for i in range(n_steps):
        x_true = x_true_mc[i+1]
        x_est, P_est = kf.predict(x_est, P_est, v_nom, w_ref_vec[i], dt, sigma_v, sigma_w)
        if gps_flags[i]:
            S = H @ P_est @ H.T + R
            innov = z_mc[i] - H @ x_est
            nu_all_ekf[j, i] = innov @ np.linalg.inv(S) @ innov
            x_est, P_est = kf.update(x_est, P_est, z_mc[i], sigma_gps)
        delta_x = x_true - x_est
        delta_x[2] = (delta_x[2] + np.pi) % (2*np.pi) - np.pi
        epsilon_all_ekf[j, i] = delta_x @ np.linalg.inv(P_est) @ delta_x

    # --- ES-EKF ---
    x_est = x0_est.copy(); P_est = P0.copy()
    for i in range(n_steps):
        x_true = x_true_mc[i+1]
        gps_flag = 1 if gps_flags[i] else 0
        z_gps = z_mc[i] if gps_flags[i] else np.zeros(2)
        x_est, P_est, nu_all_es[j, i] = es_ekf_fn(
            x_est, P_est, v_nom, w_ref_vec[i], dt,
            sigma_v, sigma_w, sigma_gps, z_gps, gps_flag)
    x_est = np.array(x_est).flatten()
    P_est = np.array(P_est)
    th = x_est[2]

```

```

R_rot = np.array([[np.cos(th), np.sin(th)], [-np.sin(th), np.cos(th)]])
dp_body = R_rot @ (x_true[:2] - x_est[:2])
dtheta = (x_true[2] - x_est[2] + np.pi) % (2*np.pi) - np.pi
eps = np.array([dp_body[0], dp_body[1], dtheta])
epsilon_all_es[j, i] = eps @ np.linalg.inv(P_est) @ eps

# --- IEKF ---
X_est = np.eye(3); P_est = P0.copy()
for i in range(n_steps):
    x_true = x_true_mc[i+1]
    gps_flag = 1 if gps_flags[i] else 0
    z_gps = z_mc[i] if gps_flags[i] else np.zeros(2)
    X_est, P_est, nis = IEKF(
        X_est, P_est, v_nom, w_ref_vec[i], dt,
        sigma_v, sigma_w, sigma_gps, z_gps, gps_flag)
    nu_all_iekf[j, i] = nis
    x_est = np.array([X_est[0,2], X_est[1,2], np.arctan2(X_est[1,0], X_est[0,0])])
    th = x_est[2]
    R_rot = np.array([[np.cos(th), np.sin(th)], [-np.sin(th), np.cos(th)]])
    dp_body = R_rot @ (x_true[:2] - x_est[:2])
    dtheta = (x_true[2] - x_est[2] + np.pi) % (2*np.pi) - np.pi
    eps = np.array([dp_body[0], dp_body[1], dtheta])
    epsilon_all_iekf[j, i] = eps @ np.linalg.inv(P_est) @ eps

epsilon_bar = np.mean(epsilon_all_ekf, axis=0)
nu_bar = np.nanmean(nu_all_ekf, axis=0)
epsilon_bar_es = np.mean(epsilon_all_es, axis=0)
nu_bar_es = np.nanmean(nu_all_es, axis=0)
epsilon_bar_iekf = np.mean(epsilon_all_iekf, axis=0)
nu_bar_iekf = np.nanmean(nu_all_iekf, axis=0)

```

C:\Users\Yahor\AppData\Local\Temp\ipykernel_45668\2578342731.py:82: RuntimeWarning:
Mean of empty slice

```
nu_bar = np.nanmean(nu_all_ekf, axis=0)
```

C:\Users\Yahor\AppData\Local\Temp\ipykernel_45668\2578342731.py:84: RuntimeWarning:
Mean of empty slice

```
nu_bar_es = np.nanmean(nu_all_es, axis=0)
```

C:\Users\Yahor\AppData\Local\Temp\ipykernel_45668\2578342731.py:86: RuntimeWarning:
Mean of empty slice

```
nu_bar_iekf = np.nanmean(nu_all_iekf, axis=0)
```

```

In [28]: nx = 3
         nz = 2

         # NEES band
         nees_lo = chi2.ppf(0.025, df=N*nx) / N
         nees_hi = chi2.ppf(0.975, df=N*nx) / N

         # NIS band
         nis_lo = chi2.ppf(0.025, df=N*nz) / N
         nis_hi = chi2.ppf(0.975, df=N*nz) / N

```

```

In [29]: t_vec = np.arange(1, n_steps + 1) * dt

         gps_mask = ~np.isnan(nu_bar)

```

```

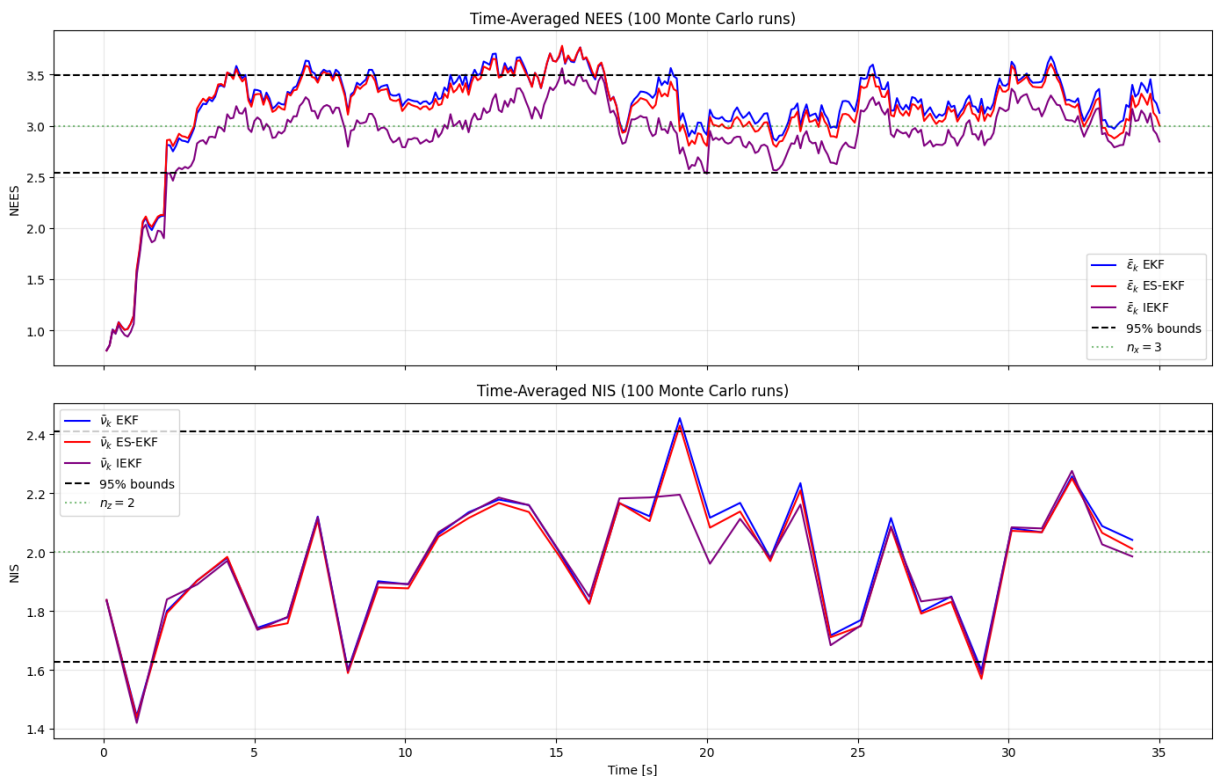
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 9), sharex=True)

ax1.plot(t_vec, epsilon_bar, color='b', linewidth=1.5, label=r'$\bar{\epsilon}_k$ EKF')
ax1.plot(t_vec, epsilon_bar_es, color='r', linewidth=1.5, label=r'$\bar{\epsilon}_k$ ES-EKF')
ax1.plot(t_vec, epsilon_bar_iekf, color='purple', linewidth=1.5, label=r'$\bar{\epsilon}_k$ IEKF')
ax1.axhline(nees_lo, color='k', linestyle='--', label='95% bounds')
ax1.axhline(nees_hi, color='k', linestyle='--')
ax1.axhline(nx, color='g', linestyle=':', alpha=0.5, label=r'$n_x = 3$')
ax1.set_ylabel('NEES')
ax1.set_title(f'Time-Averaged NEES ({N} Monte Carlo runs)')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.plot(t_vec[gps_mask], nu_bar[gps_mask], color='b', linewidth=1.5, label=r'$\bar{\nu}_k$ EKF')
ax2.plot(t_vec[gps_mask], nu_bar_es[gps_mask], color='r', linewidth=1.5, label=r'$\bar{\nu}_k$ ES-EKF')
ax2.plot(t_vec[gps_mask], nu_bar_iekf[gps_mask], color='purple', linewidth=1.5, label=r'$\bar{\nu}_k$ IEKF')
ax2.axhline(nis_lo, color='k', linestyle='--', label='95% bounds')
ax2.axhline(nis_hi, color='k', linestyle='--')
ax2.axhline(nz, color='g', linestyle=':', alpha=0.5, label=r'$n_z = 2$')
ax2.set_ylabel('NIS')
ax2.set_xlabel('Time [s]')
ax2.set_title(f'Time-Averaged NIS ({N} Monte Carlo runs)')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('NEES_NIS_comparison.png', dpi=150)
plt.show()

```



```

In [30]: nees_avg_ekf = np.mean(epsilon_bar)
         nees_avg_es = np.mean(epsilon_bar_es)
         nees_avg_iekf = np.mean(epsilon_bar_iekf)

```

```
nis_avg_ekf = np.nanmean(nu_bar)
nis_avg_es = np.nanmean(nu_bar_es)
nis_avg_iekf = np.nanmean(nu_bar_iekf)

print(f"{'Filter':<12} {'NEES':>10} {'NIS':>10}")
print("-" * 34)
print(f"{'EKF':<12} {nees_avg_ekf:>10.4f} {nis_avg_ekf:>10.4f}")
print(f"{'ES-EKF':<12} {nees_avg_es:>10.4f} {nis_avg_es:>10.4f}")
print(f"{'IEKF':<12} {nees_avg_iekf:>10.4f} {nis_avg_iekf:>10.4f}")
```

Filter	NEES	NIS
EKF	3.1928	1.9704
ES-EKF	3.1453	1.9563
IEKF	2.9028	1.9533